

Chapter 6

The MPEG-2 Video Compression Standard

6.1. INTRODUCTION

In Chapter 5 we examined the syntax of a simple MPEG-like encoder and decoder. This coder is, in fact, quite close to an MPEG-1 video encoder. In this chapter we consider the additional signal processing and syntactical elements that are used in the full MPEG-2 video standard that forms a central part of all modern digital television systems. Although the aim of the previous chapter was to provide sufficient information to allow the reader to implement an encoder and a decoder, the complexity of the full MPEG-2 video standard is such that this aim is beyond the reasonable expectations of this chapter. The chapter aims instead to give the reader a full understanding of the capabilities, power, and flexibility of the MPEG-2 video standard. Armed with a full understanding of the information provided in this chapter as well as in Chapter 5 on syntax, it should be possible for a reader to consult the relevant international standard¹ and carry out a full implementation of a compliant encoder or decoder.

After a brief introduction to the MPEG-2 video standard, we introduce the picture types used in the MPEG-2 standard. Then follows a detailed discussion of the syntax of MPEG-2. The discussion is limited to those elements of the standard that are used in current digital television applications. We then look at the video buffer verifier (VBV), which is important in ensuring the compliance of any MPEG-2 video encoder. Finally, we discuss the currently defined profiles and levels for MPEG-2.

The development of MPEG-2 followed and built on the successful earlier development of the MPEG-1 suite of standards. MPEG-1 was designed to code moving pictures and the associated audio for digital storage media applications at about 1.5 Mbit/s.² The primary limitation of MPEG-1 was that it was not designed to handle interlaced material. This does not mean that it could not code interlaced material—just that a specification that included specific coding tools to deal with

¹ ISO/IEC 13818-2.

² Although optimised for operation at around 1.5 Mbit/s, MPEG-1 can operate at rates up to 100 Mbit/s and so meet the requirements for many applications.

interlace would provide a better quality of service at a given bit rate than MPEG-1. The most important difference between MPEG-1 and MPEG-2 is the introduction of these coding tools designed to handle interlace. MPEG-2 was also designed with higher quality applications in mind than MPEG-1,³ and this led to the inclusion of a number of improved coding techniques as well as extensions to the approach used in MPEG-1. Finally, the likelihood of data errors when information is read from digital storage media is low. However, transmission errors are more common in terrestrial broadcast digital television applications. Noting this, MPEG-2 gave special consideration to error resilience in the development of the standard.

The MPEG-2 standard is designed to be extremely flexible and thus be able to handle a wide variety of possible applications. For example, the standard allows for pictures made up of more than $16,000 \times 16,000$ pixels and transmitted at bit rates in excess of 100 Gbit/s. For an encoder or a decoder to be able to claim full compliance with the standard, it would need to be able to handle these picture sizes and bit rates. For digital television applications, it is known that the picture sizes and bit rates are far less than the maximum allowed by the standard. The maximum picture size defines the size of the internal frame stores⁴ that the encoder requires whereas the bit rate defines the speed at which the decoder front end (i.e., the variable-length code (VLC) decoder) needs to operate. Requiring full compliance with the standard would therefore greatly increase the cost of encoders and, more importantly, of decoders that are required by every user.

Recognizing this problem, the committee that designed the MPEG-2 video standard has provided a number of less complex compliance points that are suitable for a wide range of applications. Using these compliance points, which are defined by profiles and levels, it is possible to design encoders and decoders that comply with the standard but require only the hardware complexity necessary to service a particular application. Further details on the currently defined profiles and levels of MPEG-2 are provided later in this chapter after the definition of the MPEG-2 syntax.

MPEG-2 video is designed to code both interlaced and progressive material. For progressive material, all information is coded as frame pictures. For interlaced material, information can be coded as a frame picture (i.e., two fields merged to form a single frame) or as two separate field pictures. The two field pictures are referred to as the top field picture and the bottom field picture. Each line of the top field picture is spatially located immediately above the corresponding line in the bottom field picture. For compatibility with interlaced displays, it is required that the bottom field picture always follows the top field picture and the top field picture always follows the bottom field picture after decoding.

The hierarchy of layers in the MPEG-2 syntax is shown in Figure 6.1. The only change in the structure used in Chapter 5 is the introduction of the group of pictures (GOP) layer. This is a compulsory layer in MPEG-1 although optional in MPEG-2. Its functions are described later in the chapter.

³ It was envisaged that a standard-definition television would require a bit rate around 4 Mbit/s and a high-definition television around 20 Mbit/s.

⁴ Although it may seem sensible to refer to these stores as picture stores, they are more commonly referred to as frame stores and so this terminology is used.

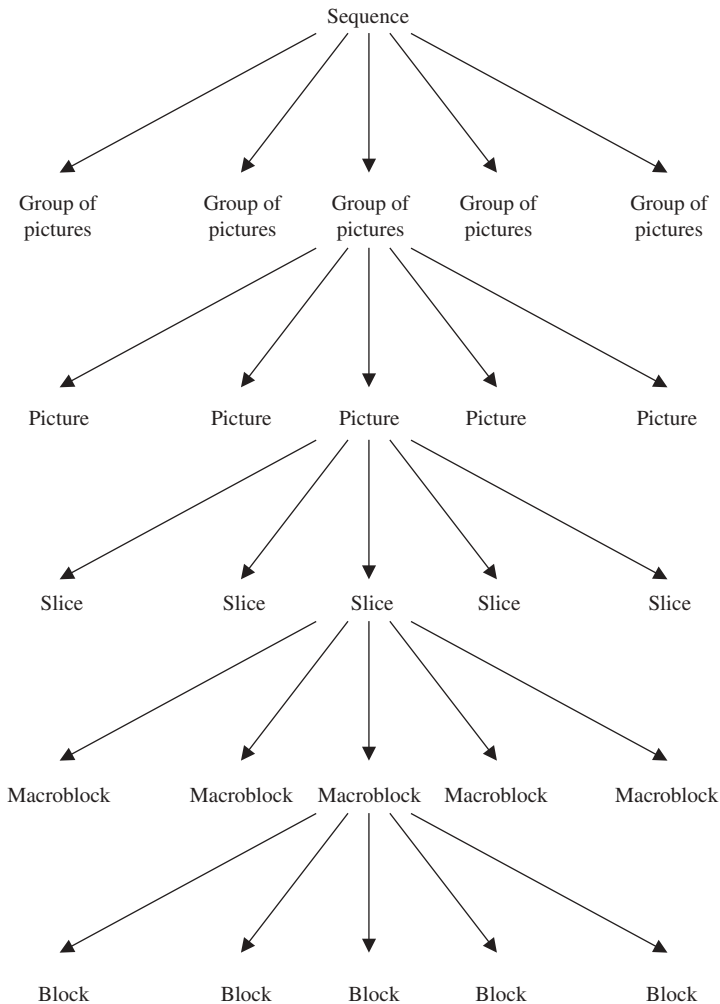


Figure 6.1 Layer hierarchy of MPEG-2 video.

6.2. PICTURE TYPES IN MPEG-2

Although a number of new coding tools are introduced in this chapter at the layer in which they appear in the syntax, there is one new coding tool that is so important for an overall understanding of the standard and has an impact at so many different levels within the syntax that it needs to be introduced immediately. This is the picture type.

In the MPEG-2 video standard, pictures are classified as one of the three types. The first of these is an *intrapicture* (I picture). In an I picture, all macroblocks are coded as *intra macroblocks* (i.e., without any prediction from any other picture).

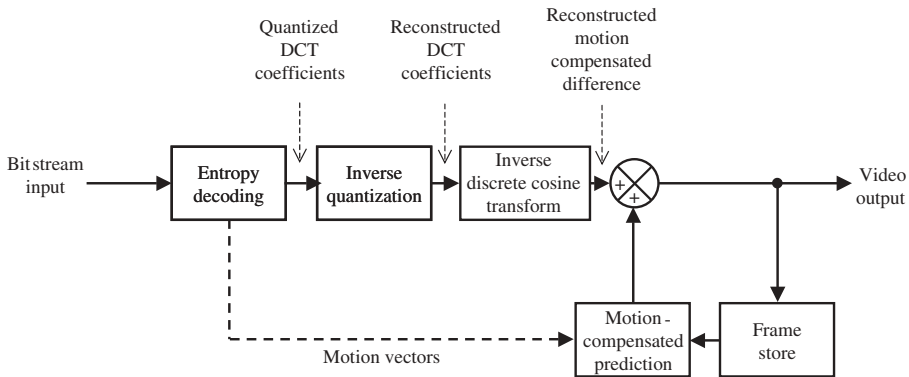


Figure 6.2 Motion-compensated DCT decoder.

This means that an I picture usually requires a large amount of data to represent it. However, in digital television applications, the regular inclusion of I pictures is essential. The reason for this becomes clear when we consider the architecture of the video decoder shown in Figure 6.2. Consider the situation when a user changes from one channel to another. For convenience, assume that the change of channel is forced to occur on a picture boundary. For the first picture after the switch, the frame store in Figure 6.2 contains a reconstruction of the last picture received from the previous channel. However, the bit stream provides coded difference information from the new channel. The video output is therefore of very poor quality, and it takes a long time before a reasonable quality video service is restored. This problem can be overcome if the first picture received from the new channel is an I picture, as I pictures can be decoded without reference to the current contents of the frame store. The decoded I picture then replaces the current contents of the frame store. Subsequent pictures can then be successfully predicted in the normal way using motion-compensated prediction.

Decoder set-top boxes are therefore invariably constrained to change to a new channel only when an I picture is being received from the new channel. This implies that I pictures need to occur regularly within the bit stream as a large I picture spacing would mean that there would be a significant delay before a change from one channel to another could occur. This would be unacceptable to users. Typically,⁵ the spacing between I pictures is around half a second. This means that a channel change can occur on average after a quarter of second and the upper threshold on the delay is half a second. Such a delay is unlikely to be noticed by users. I pictures are also useful for error resilience as they remove the effect of any transmission errors from the reconstructed picture in the frame store. As mentioned earlier, I pictures generate a large amount of data and so the inclusion of I pictures decreases the video quality of a given service bit rate.

⁵ This is not a matter of standardization but is left up to the service provider.

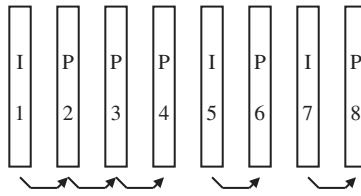


Figure 6.3 Video sequence comprising I and P pictures.

The second type of picture is the *predicted picture* (P picture). P pictures are coded using motion-compensated prediction from an immediately past I picture or an immediately past P picture. This is referred to as forward motion-compensated prediction. Although every macroblock in a P picture can be predicted using forward motion-compensated prediction, there is no requirement for every macroblock to be predicted using forward motion-compensated prediction. Macroblocks can be coded using interprediction (i.e., forming the difference between the current macroblock and the macroblock in the same position in the reference picture, which is useful when there has been little change between pictures) or intraprediction (i.e., coding without using any form of temporal prediction, which is useful for macroblocks where no suitable prediction is possible from the previous picture). A video sequence made up of I and P pictures is shown in Figure 6.3. The arrows illustrate the forward prediction. Each P picture is predicted from the I or P picture that immediately precedes it, and I pictures do not use prediction. P pictures can usually be represented by a significantly smaller amount of data than I pictures.

The final type of picture is the *bidirectionally predicted picture* (B picture). B pictures are coded using motion-compensated prediction from an immediately past I or P picture (forward prediction), an immediately future I or P picture (backward prediction), or an average of the prediction from both immediately future I or P picture and immediately past I or P picture (interpolated prediction). Thus, if three successive pictures in a video sequence are coded as an I picture followed by a B picture followed by a P picture, then the P picture is predicted from the reconstruction of the I picture and then the B picture is predicted from the reconstructions of both the I picture and the P picture. This situation is illustrated in Figure 6.4.

In Figure 6.4, the B picture (2) is predicted from the preceding I picture (1) and the following P picture (3). As motion-compensated prediction is based on reconstructed pictures, the B picture cannot be coded until after the P picture is coded. This means that the display order of the pictures (1, 2, 3) is different from the

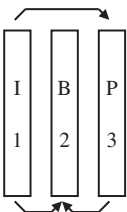


Figure 6.4 Prediction for an I picture followed by a B picture followed by a P picture.

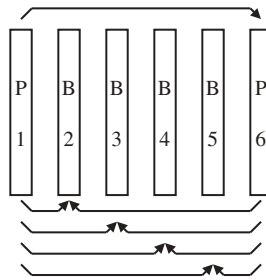


Figure 6.5 A video sequence containing several successive B pictures.

order of encoding and decoding (1, 3, 2) and that the encoder must store the original B picture until after the following P picture is coded.

The situation becomes even more complex when several B pictures lie between the I or P pictures used to form the prediction. This situation is shown in Figure 6.5, where four B pictures are predicted from a pair of P pictures.

In this case, the four pictures to be coded as B pictures would need to be stored in the encoder until the second P picture has arrived and been encoded. In digital television applications, there is only one encoder transmitting to a very large number of decoders and so the added memory requirement at the encoder is not a serious problem.

Initially, it might seem sensible to rearrange the coded pictures back into the correct display order before transmission from the encoder. For this to be done, it would be necessary for the encoder to internally store the bit-stream representation of the future reference picture (I or P) for transmission after the bit-stream representation of the B pictures. This would add slightly to encoder complexity.

At the decoder, if the coded pictures are transmitted in their original (i.e., display) order, then the picture (I or P) preceding the B pictures is received and can be immediately decoded as its reference picture (a previous I or P picture) occurred earlier in the bit stream and so has already been received by the decoder. The data representing the four B pictures need to be held at the decoder until after the arrival of the second reference picture (again I or P). After that reference picture has been decoded, the four B pictures that preceded it can finally be decoded.

However, there is a simpler approach that can be employed, which eases the task for both the encoder and the decoder: Pictures are transmitted in the same order that they are encoded. The encoder has to wait for the arrival of the second reference picture (I or P) before it can encode any B pictures that precede it. When the second reference picture is encoded, then it is transmitted immediately followed by the encoded B pictures that occurred immediately before it. This means that the encoder no longer needs to store the coded bit-stream representation of the future reference picture.

When a B picture arrives at the decoder, both of the reference pictures needed for its prediction have already arrived. Thus, when a coded B picture arrives, it can immediately be decoded. There is never a need to store the data representing a B picture at the decoder while a later reference picture is awaited. This means that for video sequences employing B pictures, the picture transmission order is different from the picture display order. The picture order rearrangement done by the encoder

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
I	B	B	P	B	B	P	B	B	P	B	B	P	B	B	I

Figure 6.6 Pictures to be transmitted in display order with picture types shown.

is easily undone by the decoder, thereby ensuring that pictures are always displayed in the correct order.

Consider the case of 16 pictures that are to be coded with the picture types shown in Figure 6.6.

The transmission order after encoding is shown in Figure 6.7. The different sizes used for each picture are to indicate that I pictures usually require the largest amount of data to represent them, whereas P pictures require less data and B pictures the smallest amount of data.

The decoder requires just two frame stores (each able to store one decoded picture) irrespective of the number of consecutive B pictures that occur in the video sequence. The decoding procedure is shown in Table 6.1.

Every time an I or a P picture arrives at the decoder, it is decoded and overwrites the oldest I or P picture currently stored in the pair of frame stores. When a B picture arrives, the two reference pictures from which it was predicted are already available in the decoder's frame stores and so the B picture can be immediately decoded. The process of reordering the received pictures into the correct order for display is therefore straightforward.

The use of B pictures greatly increases the overall coding efficiency since motion-compensated prediction is almost always possible. There are a number of situations where forward-only motion compensation does not work effectively. We now consider how the use of bidirectional motion-compensated prediction effects these situations.

Forward-only motion-compensated prediction is ineffective at a scene cut because the reference picture bears little or no resemblance to the current picture to be coded. With bidirectional prediction, the reference picture used for forward prediction is still of little use. However, the picture used for backward prediction is highly likely to be suitable for motion-compensated prediction.

When a foreground object moves and uncovers a previously hidden background material, forward-only motion-compensated prediction is ineffective since a match to the uncovered background cannot be found in the past reference picture. With bi-directional motion-compensated prediction, performance is improved, as it is likely that the background is still uncovered in the future reference picture.

I	P	B	B	P	B	B	P	B	B	P	B	B	I	B	B
1	4	2	3	7	5	6	10	8	9	13	11	12	16	14	15

Figure 6.7 Transmission order of pictures shown in Figure 6.6.

Table 6.1 Decoding process for bit stream of Figure 6.6.

Picture decoded and picture type	Contents of frame store 1	Contents of frame store 2	Picture to be displayed
1 (I)	1	—	
4 (P)	1	4	1
2 (B)	1	4	2
3 (B)	1	4	3
7 (P)	7	4	4
5 (B)	7	4	5
6 (B)	7	4	6
10 (P)	7	10	7
8 (B)	7	10	8
9 (B)	7	10	9
13 (P)	13	10	10
11 (B)	13	10	11
12 (B)	13	10	12
16 (I)	13	16	13
14 (B)	13	16	14
15 (B)	13	16	15

When a new material enters a picture (e.g., a new object or new background due for example to camera movement), forward-only motion-compensated prediction is ineffective in predicting this new material since it is not available in the past reference picture. With bidirectional motion-compensated prediction, performance improves, as it is likely that the new material can still be seen in the future reference picture.

If an object moves between pictures by an amount greater than the search area, then forward-only motion-compensated prediction is not effective. Unless the object slows down, this problem also exists for bidirectional motion-compensated prediction and can only be addressed by increasing the size of the search area.

A further advantage of B pictures is that they are not used to predict other pictures in the video stream. This means that if a B picture is quantized coarsely, it will have no impact on the efficiency of coding the subsequent pictures in the sequence. Coding I pictures or P pictures coarsely would mean that the subsequent pictures would likely require additional bits as the quality of the motion-compensated prediction from the coarsely coded I picture or P picture would be reduced.

It is clear that bidirectional motion-compensated prediction can improve coding performance in a number of situations. It needs to be remembered that these improvements do come with some significant costs. The first of these costs is that all pictures to be encoded using motion-compensated prediction (i.e., P or B pictures) and the relevant reference picture or pictures are often separated by several picture times when B pictures are used. This means that the moving objects have moved by a greater amount than would have been the case if the current and reference pictures were adjacent. Larger search areas are therefore required with a consequential increase in computational requirements at the encoder. The wider spacing also leads to a decrease

in coding efficiency. This is a particular problem for P pictures since the separation between a P picture and its reference picture is greater than the separation between an associated B picture and its reference pictures. This is shown in Figure 6.6.

In addition, for B pictures, motion-compensated prediction needs to be performed twice—once in the forward direction and once in the backward direction. A decision also needs to be made as to whether to use forward prediction, backward prediction, or an average of the two to code the current macroblock. This decision needs to be signaled to the decoder together with the appropriate motion vector(s). If the average of the two predictions is to be used, then two motion vectors need to be transmitted.

A further problem introduced by the use of B pictures is an increase in delay. This problem arises because a B picture cannot be transmitted until the future I or P picture from which it is to be predicted has arrived and been encoded. If there are several consecutive B pictures, there can be several picture times. Although not a problem for distribution television services, this can be a serious problem in two-way interpersonal communication services where round trip delay is very important. For these latter applications, B pictures are often not used.

6.3. THE SYNTAX OF MPEG-2

We now discuss the special coding tools and semantic definitions that are used in the full MPEG-2 video coding standard. These build directly on the tools and semantics described in Chapter 5. As in the previous chapter, our discussions are ordered by the structure of the video bit stream.

Like the syntax described in Chapter 5, MPEG-2 makes use of start codes to allow resynchronization within the bit stream. Each start code begins with the 24-bit value 0x000001 followed by the appropriate start-code type as shown in Table 6.2.

Table 6.2 MPEG-2 start codes.

Type of start code	Start-code values
picture_start_code	0x00
slice_start_code	0x01 to 0xAF
Reserved	0xB0 to 0xB1
user_data_start_code	0xB2
sequence_header_code	0xB3
sequence_error_code	0xB4
extension_start_code	0xB5
Reserved	0xB6
sequence_end_code	0xB7
group_start_code	0xB8
Reserved	0xB9 to 0xFF ^a

^a These start code values are used in MPEG-2 system specification.

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

MPEG-2 also makes use of extension start codes to expand the number of headers available. Before moving to a full discussion of the syntax, we first introduce the definition and functions of extension start codes.

6.3.1. Extension Start Code and Extension Data

Extension start codes identify extensions of the video syntax beyond those elements that were provided in the MPEG-1 video syntax. Like all start codes, extension start codes begin with the 24-bit pattern 0x000001 followed by the 8-bit start-code identifier 0xB5. The type of extension is identified by a 4-bit **extension_start_code_identifier** field that immediately follows the extension start code and is defined in Table 6.3.

Extensions considered in this chapter include sequence extension, sequence display extension, quantizer matrix extension, copyright extension, picture display extension, and picture coding extension. Other currently defined extensions are provided for the various scalable⁶ modes of MPEG-2. As these are not used in any current digital television applications, they are not considered in detail here.

We are now in a position to discuss the MPEG-2 syntax. We begin at the top layer called the sequence layer and then move through the group of pictures, picture,

Table 6.3 Codes for **extension_start_code_identifier**.

extension_start_code_identifier	Extension data type
0000	Reserved
0001	Sequence extension
0010	Sequence display extension
0011	Quantizer matrix extension
0100	Copyright extension
0101	Sequence scalable extension
0110	Reserved
0111	Picture display extension
1000	Picture coding extension
1001	Picture spatial scalable extension
1010	Picture temporal scalable extension
1011	Reserved
1100	Reserved
1101	Reserved
1110	Reserved
1111	Reserved

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

⁶ Scalability in video compression is the ability to code a sequence into several bit streams. Decoding just the base layer bit stream gives a basic service whereas decoding the base layer bit stream plus one or more of the enhancement bit streams provides an improvement in the received service (e.g., higher quality, higher spatial resolution, higher frame rate).

slice, macroblock, and block levels. Notice that the MPEG-2 syntax is really just a superset of the syntax described in Chapter 5.

6.3.2. Sequence Layer

The complete syntax for a video sequence in MPEG-2 is shown in Table 6.4.

For a new video sequence, the first step is to search for a **start_code** (line 2) that should be a `sequence_header()` (line 3) as discussed in Section 6.3.2.1. If this is followed by an **extension_start_code** (line 4), then the bit stream is compliant with the MPEG-2 video standard. Otherwise, it is compliant with the MPEG-1 video standard (line 24).⁷ For MPEG-2 video, the next item in the bit stream is the `sequence_extension()` (line 5) (Section 6.3.2.2). This is followed by `extension_and_user_data(0)`

Table 6.4 Video sequence syntax for MPEG-2 video.

	Syntax	Bits	Mnemonic
1	<code>video_sequence() {</code>		
2	<code>next_start_code()</code>		
3	<code>sequence_header()</code>		
4	<code>if(nextbits() == extension_start_code) {</code>		
5	<code>sequence_extension()</code>		
6	<code>do {</code>		
7	<code>extension_and_user_data(0)</code>		
8	<code>do {</code>		
9	<code>if(nextbits() == group_start_code) {</code>		
10	<code>group_of_pictures_header()</code>		
11	<code>extension_and_user_data(1)</code>		
12	<code>}</code>		
13	<code>picture_header()</code>		
14	<code>picture_coding_extension()</code>		
15	<code>extension_and_user_data(2)</code>		
16	<code>picture_data()</code>		
17	<code>} while ((nextbits() == picture_start_code) (nextbits() ==</code>		
	<code>group_start_code))</code>		
18	<code>if (nextbits() != sequence_end_code) {</code>		
19	<code>sequence_header()</code>		
20	<code>sequence_extension()</code>		
21	<code>}</code>		
22	<code>} while (nextbits() != sequence_end_code)</code>		
23	<code>} else {</code>		
24	<code>/* ISO/IEC 11172-2 MPEG-1*/</code>		
25	<code>}</code>		
26	<code>sequence_end_code</code>	32	bslbf
27	<code>}</code>		

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

⁷ All MPEG-2 video decoders need to be able to decode MPEG-1 encoded video (ISO/IEC 11172-2).

(line 7), which can include user data as discussed in Section 6.3.2.3 or a `sequence_display_extension()` as discussed in Section 6.3.2.4. Next in the bit stream there can be a **group_start_code** indicating the presence of a `group_of_pictures_header()` followed by `extension_and_user(1)` (lines 10–11) as discussed in Sections 6.3.3.1 and 6.3.3.2. With or without the group of pictures information, next follows `picture_header()` (Section 6.3.4.1), `picture_coding_extension()` (Section 6.3.4.2), `extension_and_user_data(2)` (Sections 6.3.4.3–6.3.4.5) followed by the coded slice and macroblock information in `picture_data()` (lines 13–16) (Sections 6.3.5–6.3.7).

Groups of pictures and pictures continue until a start code arrives, which is neither a **group_start_code** nor a **picture_start_code** (line 17). At this point, if the next start code is not a **sequence_end_code**, then a new `sequence_header()` and `sequence_header_extension()` are read (lines 18–20) and the loop starting at line 6 is repeated. Alternatively, a **sequence_end_code** occurs (line 26) indicating the end of the current sequence. As the chapter unfolds, we look in more detail at each of these major structures as well as the elements that make them up.

6.3.2.1. The Sequence Header

In MPEG-2, information about a video sequence is contained in the `sequence_header()`, the `sequence_extension()`, the `sequence_display_extension()`, and the `extension_and_user_data(0)`.⁸ The `sequence_header()` is designed to be similar to the sequence header used in MPEG-1. The additional information required for MPEG-2 sequences (e.g., greater number of pixels in a line or lines in a picture, higher bit rates, wider range of frame rates) is achieved by including extension data in the `sequence_extension()` header with the least significant bits (LSB) of the parameter contained in the `sequence_header()` and the most significant bits (MSB) in the `sequence_extension()`. For convenience, in this section we consider those parts of the `sequence_extension()` that augment values in the `sequence_header()`.

The next section on the `sequence_extension()` will only cover the new elements contained in the `sequence_extension()`.

The `sequence_header()` begins with a **sequence_header_code**, which is a 32-bit start code. Other parameters fully or partly defined in the `sequence_header()` are listed below.

Horizontal and Vertical Picture Size The picture size in pixels is defined by the variables **horizontal_size** and **vertical_size**. Both are 14-bit values of which the 12 LSB (**horizontal_size_value** and **vertical_size_value**) are contained in the `sequence_header()` and the two MSB are contained in the `sequence_extension()`. The maximum allowed horizontal and vertical picture sizes for pictures in MPEG-2 are thus in excess of 16,000 pixels. In order to avoid start-code emulation, the values of **horizontal_size_value** and **vertical_size_value** cannot be zero. This implies

⁸ This structure has been used to maintain compatibility with MPEG-1 standard that has only a sequence header.

that values of **horizontal_size** and **vertical_size** that are multiples of 4096 are not allowed.

Picture Aspect Ratio Aspect ratio information is identical to that defined in Chapter 5.

Frame Rate The 4-bit **frame_rate_code** in the sequence header serves the same purpose as described in Chapter 5 in defining the **frame_rate_value**. In the full MPEG-2 standard, the range of possible frame rates can be expanded using the 2-bit **frame_rate_extension_n** field (range 0–3) and the 5-bit **frame_rate_extension_d** field (range 0–31) defined in the `sequence_extension()`. The equation below shows how these can be used to vary the frame rate.

$$\text{frame_rate} = \text{frame_rate_value} \times \frac{\text{frame_rate_extension_n} + 1}{\text{frame_rate_extension_d} + 1}$$

Although a part of the full MPEG-2 specification, the existing profiles do not make use of **frame_rate_extension_n** and **frame_rate_extension_d**, both of which must take the value zero.

Sequence Bit Rate The bit rate of the sequence is defined by a 30-bit value **bit_rate** with the 18 LSB (**bit_rate_value**) in the `sequence_header()` and the 12 MSB (**bit_rate_extension**) in the `sequence_extension()`. The bit rate is defined in multiples of 400 bits/s. It is therefore possible to specify bit rates in excess of 400 Gbits/s for MPEG-2 video bit streams. The variable **bit_rate** may not take the value zero and so the allowed range is $400\text{--}400 \times (2^{30} - 1)$ bits/s.

Video Buffer Verifier Buffer Size The **vbv_buffer_size** variable defines the size of the video buffer verifier rate control buffer. The role of this buffer is described further in Section 6.4. It is an 18-bit field with the 10 LSB (**vbv_buffer_size_value**) in the `sequence_header()` and the eight MSB (**vbv_buffer_size_extension**) in the `sequence_extension()`. It specifies the minimum size rate control buffer needed by a decoder to be able to successfully decode the video sequence. The buffer size in bits is derived from **vbv_buffer_size** according to the equation given below.

$$\text{VBV_BufferSize} = 16 \times 1024 \times \text{vbv_buffer_size}$$

The larger the buffer, the better the smoothing and the higher the video quality for a given bit rate. However, both delay and cost increase as the buffer size increases. The maximum buffer size is in excess of 4 Gbits (range 0 to $16 \times 1024 \times (2^{18} - 1)$).

Constrained Parameter Flag The constrained parameter flag was used in the specification of MPEG-1 to perform a limited but similar function to that achieved by the definition of levels in MPEG-2. For MPEG-2 video bit streams, it must be set to zero.

8	16	19	22	26	27	29	34
16	16	22	24	27	29	34	37
19	22	26	27	29	34	34	38
22	22	26	27	29	34	37	40
22	26	27	29	32	35	40	48
26	27	29	32	35	40	48	58
26	27	29	34	38	46	56	69
27	29	35	38	46	56	69	83

Figure 6.8 Default quantizer matrix for intrablocks. © This Figure is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Quantizer Matrix Definition In the MPEG video compression standards, DCT coefficients can be quantized using different quantizers depending upon their relative frequency using the method described in Section 4.4.3. This allows the quantization to better match the characteristics of the human visual system. The quantizer matrix definition allows the user to specify how this quantization is performed.

The quantizer matrices given in Figures 6.8 and 6.9 are the defaults defined in the standard for intrablocks and inter (including motion-compensated) blocks, respectively. They are used if no other quantizer matrices are specified in the bit stream and apply equally to luminance and chrominance blocks. However, it is possible to download new quantizer matrices both at the sequence and picture layers. At the sequence layer, a 1-bit flag is used to indicate whether a new intrablock quantizer matrix or a new interblock quantizer matrix is to be transmitted as part of the sequence header. If a flag is set to one, it is immediately followed by the new quantizer matrix represented by sixty-four 8-bit numbers ordered in the default zig-zag scan order used for

16	16	16	16	16	16	16	16
16	16	16	16	16	16	16	16
16	16	16	16	16	16	16	16
16	16	16	16	16	16	16	16
16	16	16	16	16	16	16	16
16	16	16	16	16	16	16	16
16	16	16	16	16	16	16	16
16	16	16	16	16	16	16	16

Figure 6.9 Default quantizer matrix for interblocks. © This Figure is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 6.5 Profile indication.

Profile indication	Profile
111	Reserved
110	Reserved
101	Simple
100	Main
011	SNR scalable
010	Spatially scalable
001	High
000	Reserved

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

DCT coefficients. This means that the values in the quantizer matrix need to be in the range of 1–255. A zero quantizer value is, of course, not allowed. Any quantizer matrix downloaded at the sequence layer applies to both luminance and chrominance blocks.

6.3.2.2. The Sequence Extension

As already described in Section 6.3.2.1, a number of parameters in the `sequence_header()` are defined by entries in both the `sequence_header()` and the `sequence_extension()`. Several other parameters are defined only in the `sequence_extension()` and so are relevant to MPEG-2 video bit streams but not to MPEG-1 video bit streams.

Profile and Level Indication An 8-bit **profile_and_level_indication** field defines the profile and level of the current video sequence. For profiles and levels⁹ defined in the original MPEG-2 standard, which includes all of the profiles and levels used in digital television applications, the MSB in this field (called an escape bit) is 0 and is followed by a 3-bit profile indication and finally a 4-bit level indication. The profile indication field is defined in Table 6.5, whereas the level indication is shown in Table 6.6. Other profiles have been defined subsequently and have been defined with the escape bit set to 1. We will not consider these further here as they do not relate to digital television services. Further discussion of currently defined profiles and levels is included later in this chapter.

Progressive Sequence The 1-bit **progressive_sequence** field defines that the sequence is progressive (i.e., each picture is fully imaged at the same time) when set to one. Otherwise, the sequence is interlaced (i.e., each picture is imaged twice to produce two fields separated in time by half the period between pictures). Most current digital television applications use interlaced sequences.

⁹ Profiles define a subset of the complete syntax of the standard, whereas levels place constraints on the values that may be taken by parameters in the bit stream. See Section 6.5 for more details.

Table 6.6 Level indication.

Level indication	Level
1011 to 1111	Reserved
1010	Low
1001	Reserved
1000	Main
0111	Reserved
0110	High 1440
0101	Reserved
0100	High
0000 to 0011	Reserved

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Chrominance Format MPEG-2 allows three chrominance formats: 4:2:0, 4:2:2, and 4:4:4. The format used in the current video sequence is signaled in the 2-bit **chroma_format** field in the `sequence_extension()`. All current distribution digital television applications use the 4:2:0 chrominance format and so this field takes the value 01.

Low Delay The 1-bit **low_delay** flag in the sequence extension indicates that the video sequence is a low-delay sequence and therefore does not contain B pictures that, as explained earlier, introduce delay due to the need for picture reordering after decoding.

6.3.2.3. User Data at the Sequence Layer

User data can be specified at the sequence layer and is preceded by a start code and a unique start-code value as shown in Table 6.1. Then follows the user data in 8-bit bytes until a new start code is encountered. Because start-code emulation must be avoided, user data must not contain a string of 23 or more consecutive zeros. As the name implies, user data is user defined for particular applications. As is shown later, user data can also be present at lower levels of the syntax.

6.3.2.4. The Sequence Display Extension

The final type of information that is available at the sequence layer is the `sequence_display_extension()`. It is important to realize that the MPEG-2 video specification does not define the display process. Thus, the information contained in the `sequence_display_extension()` may be helpful to some decoders while it may be completely ignored by others. All compliant decoders must be capable of decoding bit streams in which these values are defined. The elements contained in the `sequence_display_extension()` are described briefly below.

Video Format The 3-bit **video_format** field indicates the format of the analog video before coding. Possible formats include component, PAL, NTSC, SECAM, and MAC.

Color Description The 1-bit **color_description** flag when set to one indicates the presence of the following three elements next in the bit stream:

- **color_primaries**: an 8-bit number that defines the chromaticity coordinates of the source primaries.
- **transfer_characteristics**: an 8-bit number that specifies the optoelectronic characteristics of the source picture. This is commonly referred to as the gamma.
- **matrix_coefficients**: an 8-bit number that specifies the matrix coefficients used to derive the luminance and chrominance signals from the red, green, and blue primaries in the original source material.

Display Horizontal and Vertical Size The two 14-bit parameters **display_horizontal_size** and **display_vertical_size** define a rectangle that may be considered as the active region of the intended display. If the rectangle is smaller than the decoded picture size, the display would be expected to show only a part of the decoded picture. Alternatively, if the rectangle is larger than the decoded picture size, the display would be expected to show the decoded picture in a portion rather than all of the display.

6.3.3. The Group of Pictures Layer

A *group of pictures* is defined as one or more coded pictures with the first coded picture (in transmission order) as an intracoded picture. The situation is illustrated in Figure 6.10, which shows the position of the `group_of_pictures_header()` for a sequence in both transmission and display order.

A GOP can contain more than one I picture as well as any number of P pictures and B pictures. The last coded picture (in display order) of a GOP is either an I picture or a P picture. The GOP layer provides a convenient structure for random access into a video sequence. It was developed as part of the MPEG-1 video standard that was designed to handle video from digital storage media such as CDs. Clearly, random access to the video material is essential for such applications. In the MPEG-2 video standard, the use of the group of pictures layer is optional.

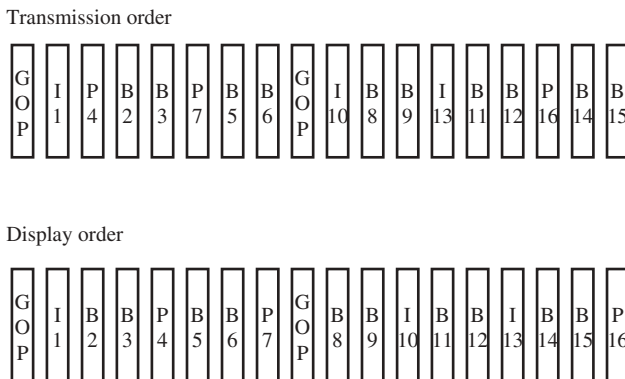


Figure 6.10 Positioning of the group of pictures header.

6.3.3.1. Group of Pictures Header

The `group_of_pictures()` header begins with a **group_start_code** as defined in Table 6.2. Other items defined in the `group_of_pictures_header()` are listed below.

Time Code The 25-bit **time_code** field is in the format used for the transmission of time and control codes for video tape recorders.¹⁰ The information carried in this field plays no part in the video decoding process.

Closed GOP and Broken Link Flags As shown in Figure 6.10, in transmission order the first I picture in a GOP may be followed by a number of B pictures calculated using a reference picture in the previous GOP. In general, these B pictures need both the I picture in the current GOP and the last I picture or P picture from the previous GOP for correct decoding. In television applications, independent pieces of video material are often edited together. As a result of this editing, the previous GOP may no longer be present in the bit stream having been replaced by a GOP from a different sequence. The 1-bit **closed_gop** flag indicates that the B pictures that follow the first I picture in the GOP have been encoded in intramode or using motion-compensated prediction only from the first I picture of the GOP. They can therefore still be correctly decoded even if the previous GOP is no longer available in the bit stream. The 1-bit **broken_link** flag indicates that the B pictures following the first I picture in the GOP cannot be correctly decoded. It is set to one during the editing process if the previous GOP is removed unless the current GOP is a closed GOP.

6.3.3.2. User Data

User data can also be included in the GOP layer. It begins with the unique **user_data_start_code** and has a format identical to that used for user data at the sequence layer.

6.3.4. The Picture Layer

The picture layer includes the `picture_header()` and the `picture_coding_extension()`. It may also contain the optional extensions `quant_matrix_extension()`, `picture_display_extension()`, and `copyright_extension()`.¹¹ The picture layer is organized in a similar manner to the sequence layer in that the `picture_header()` is almost identical to MPEG-1, whereas new additions at the picture layer required for MPEG-2 are contained in the `picture_coding_extension()`. The picture header begins with the 32-bit **picture_start_code**. Other items defined at the picture layer are listed below.

¹⁰ See IEC Publication 461, Time and Control Code for Video Tape Recorders.

¹¹ Extensions for temporal scalability and spatial scalability can also occur at the picture layer. As these are not used for current distribution television applications, they are not considered here.

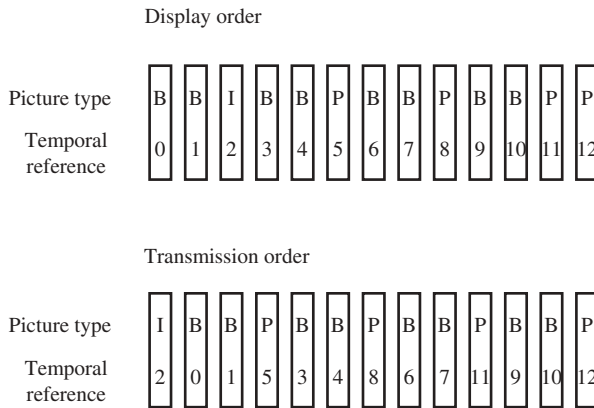


Figure 6.11 Use of the **temporal_reference** in display and transmission orders.

6.3.4.1. Picture Header

In this section we discuss parameters defined in the `picture_header()`.

Temporal Reference The **temporal_reference** is a 10-bit integer that is associated with each coded picture and defines the display order of the pictures. This information can be useful to a decoder as pictures do not necessarily arrive at the decoder in display order when B pictures are used. The first picture in display order in each group of pictures has a **temporal_reference** value of zero, and the temporal reference is incremented as described below for each successive picture in the group of pictures.

For sequences that are not coded in low-delay mode (i.e., **low_delay** at the sequence layer is set to zero),¹² the **temporal_reference** for each coded picture increases by one modulo 1024 in display order. For a frame picture coded as two field pictures, as is allowed for interlaced sequences, each field is assigned the same temporal reference. Figure 6.11 shows the use of the temporal reference in a typical group of pictures where each picture is a frame picture. Both display and transmission orders are shown.

Picture Coding Type The 3-bit **picture_coding_type** field specifies the prediction method used to code a particular picture as defined by Table 6.7.

Although every macroblock in an I picture is coded in intramode, it is still possible to have intracoded macroblocks in both P pictures and B pictures. In addition and as described earlier, not every macroblock in a B picture needs to be coded using bidirectional prediction, as prediction from only one of the two reference pictures is also possible.

VBV Delay The *video buffer verifier* is a hypothetical decoder that is connected directly to the output of the encoder. Its role is to ensure that buffer overflow or

¹² Distribution digital television services do not use low-delay mode and so the case of low delay mode is not considered. Details can be found in the international standard.

Table 6.7 Definition of `picture_coding_type`.

<code>picture_coding_type</code>	Prediction method used for coding
000	Forbidden
001	Intrapicture (I)
010	Predictive picture (P)
011	Bidirectionally predicted picture (B)
100	Not to be used ^a
101–111	Reserved

^a This type was used for D pictures (pictures that only contained the DC DCT coefficient after coding in intramode) in MPEG-1. It is not allowed in MPEG-2.

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

buffer underflow does not occur at the decoder. The VBV is described in detail later in this chapter. The `vbv_delay` field sets the number of periods of a 90-kHz clock (which is derived from the 27-MHz system clock¹³) that the VBV waits after the receipt of the final byte of `picture_start_code` before decoding the picture. The `vbv_delay` can be used by the decoder to correct for any variation between the timing (such as the clock speeds) at the encoder and the decoder.

Redundant Fields The picture header next contains four redundant fields, namely, `full_pel_forward_vector` (1 bit), `forward_f_code` (3 bits), `full_pel_backward_vector` (1 bit), and `backward_f_code` (3 bits). All were used in the picture header for the MPEG-1 video standard to specify information about the way that motion vectors had been coded. They are not used in MPEG-2 and are set to predefined values (the 1-bit fields to 0 and the 3-bit fields to 111). Instead, data on the coding of motion vectors are included in the `picture_coding_extension()` for MPEG-2 bit streams.

Extra Picture Information The 1-bit `extra_bit_picture` field when set to one is followed immediately by the 8-bit `extra_information_picture` field. This field can be used for future extensions of the standard. It is currently not used and must not be used in compliant bit streams. After this field, the `extra_bit_picture` field is again checked and if set to one, a further 8-bit `extra_information_picture` field is read. This continues until `extra_bit_picture` is not set to one. If the `extra_bit_picture` is initially zero, then no `extra_information_picture` data is read.

6.3.4.2. Picture Coding Extension

The `picture_coding_extension()` begins with a 32-bit `extension_start_code` and a 4-bit `extension_start_code_identifier` as defined in Table 6.3. Other parameters are defined below.

¹³ See Chapter 11 on MPEG-2 systems for further details.

Motion Vector Range Motion vector ranges are defined by the values of the four fields **f_code[0][0]**, **f_code[0][1]**, **f_code[1][0]**, and **f_code[1][1]** that refer to forward horizontal, forward vertical, backward horizontal, and backward vertical motion vector components, respectively. Each field is of length 4-bits, and not all fields are relevant for every picture type. Thus, no **f_code[][]** fields are needed in an I picture,¹⁴ and so all 4-bit fields take the value 1111. Only forward **f_code[][]** fields are required for P pictures, and so the two 4-bit backward fields take the value 1111.

The four values are needed to allow a separate **f_code[][]** value for forward and backward motion vectors (both of which could be required for B pictures) and for the vertical and horizontal component of each motion vector. The **f_code[][]** value defines the motion vector range as defined in Table 6.8. The range for the vertical component of field motion vectors in frame pictures is half of that for all other components. This is because a one-pixel vertical displacement in a field picture is equivalent to a two-pixel horizontal displacement in a frame picture because a line from the alternate field lies between the position of the current pixel and the pixel displaced by one pixel vertically in the same field. A smaller motion vector range is therefore appropriate.

From the data in Table 6.8, it can be seen that very large motion vector ranges are allowed and that motion-compensated prediction is performed to half pixel accuracy.

Intra-DC Precision In an intracoded macroblock, the value of the DC DCT coefficient for luminance and chrominance defines the average color of the macroblock. Coding this information with insufficient accuracy means that only a limited range of colors are possible. This can show up as blocking artifacts in flat regions of a picture.

Table 6.8 Motion vector ranges for various values of **f_code**.

f_code[][] value	Range for vertical component of field motion vectors in frame pictures	Range for all other motion vectors
0	Not allowed	
1	−4 to +3.5	−8 to +7.5
2	−8 to +7.5	−16 to +15.1
3	−16 to +15.5	−32 to +31.5
4	−32 to +31.5	−64 to +63.5
5	−64 to +63.5	−128 to +127.5
6	−128 to +127.5	−256 to +255.5
7	−256 to +255.5	−512 to +511.5
8	−512 to +511.5	−1024 to +1023.5
9	−1024 to +1023.5	−2048 to +2047.5
10–14	Reserved	Reserved
15	Used when a particular f_code[][] is not needed	

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

¹⁴Unless the I picture makes use of concealment motion vectors for error concealment. These are described later in the `picture_coding_extension()`.

Table 6.9 Definition of **intra_dc_precision**.

intra_dc_precision	Precision of DC DCT coefficient in I blocks (bits)
00	8
01	9
10	10
11	11

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

For this reason, particular care is taken in coding these DC DCT coefficients using 8, 9, 10, or 11-bit precision. This precision can be set at the picture layer using the 2-bit **intra_dc_precision** field defined in Table 6.9.

Picture Structure The field or frame structure of a picture is defined by the 2-bit **picture_structure** field as defined in Table 6.10.

Top Field First This **top_field_first** flag has a number of meanings, depending upon the values of the **progressive_sequence** flag (sequence layer) and the **picture_structure** and **repeat_first_field** flags (picture layer).

For interlaced sequences (**progressive_sequence** set to zero), this flag indicates which field is to be output first by the decoder. For field pictures, it always takes the value zero since whether the field is the top field or the bottom field is indicated by the **picture_structure** field. For frame pictures, if **top_field_first** is set to one then the first field output from the decoding process is the top field. Otherwise, the first field output is the bottom field.

For progressive sequences (**progressive_sequence** set to one), the **top_field_first** and **repeat_first_field** flags indicate how many times (one, two, or three) the reconstructed progressive picture is to be output by the decoding process as defined in Table 6.11.

Frame-Only Prediction and DCT Frame pictures in MPEG-2 can still be predicted using field prediction. In addition, the DCT can also be carried out in either field or frame mode. Both of these are discussed further at the macroblock level later

Table 6.10 Definition of **picture_structure**.

picture_structure	Type of picture
00	Reserved
01	Top field of a pair of field pictures
10	Bottom field of a pair of field pictures
11	Frame picture

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 6.11 Use of **top_field_first** and **repeat_first_field** for progressive sequences.

top_field_first	repeat_first_field	Decoder output
0	0	One progressive picture
0	1	Two identical progressive pictures
1	0	Not allowed
1	1	Three identical progressive pictures

in this chapter. However, if the flag **frame_pred_frame_dct** is set to one in a frame picture from an interlaced video sequence, then only frame prediction and frame DCT are used in the coding of this picture. This allows some savings in overhead bits to be achieved at the macroblock layer. The flag is set to zero in field pictures since field prediction is always employed. It is always set to one in progressive sequences since only frame prediction and frame DCT are employed.

Concealment Motion Vectors The MPEG-2 syntax allows motion vectors to be transmitted in I macroblocks even though motion-compensated prediction is not used for the coding of these macroblocks. The availability of these motion vectors allows improved concealment if transmission errors occur. Transmitting the motion vector in the same macroblock which has the data for that macroblock is of little use because if the data is lost, then it is highly likely that the motion vector is also lost. For this reason, a concealment motion vector sent in a particular macroblock refers to the macroblock directly below the current macroblock. For the last row of macroblocks in a picture, concealment motion vectors serve no useful purpose and so should be set to zero to reduce unnecessary overhead.

Quantizer Scale Type The quantizer to be used for the coding and decoding of DCT coefficients is defined by a 5-bit **quantizer_scale_code** field, which is defined at the slice and macroblock layers. This can be decoded in one of the two ways depending on the value of the **q_scale_type** flag as defined in Table 6.12.

The first column of quantizer values is the set used in MPEG-1 with the quantizer directly proportional to the **quantizer_scale_code** value. The second column is an alternative set introduced for MPEG-2, which allows both an increased range of possible quantizers (1–112 as opposed to 2–62) as well as a large range of small quantizer values (i.e., 1–8 in steps of 1), which is useful for high-quality applications.

Intra-Variable-Length Code Format The variable-length code words used to represent run-level pairs of DCT coefficients are the same as those given in Chapter 5, with the exception that if the **intra_vlc_format** flag is set then an alternate set of variable-length code words are used for the coding of intramacroblocks. Since intra macroblocks can occur in any type of picture (I, P, or B), the variable-length code word table to be used needs to be specified for every picture. The exact specification of the alternate table can be found in the standard.

Table 6.12 Definition of quantizer step size for different values of **q_scale_type**.

quantizer_scale_code	Quantizer step size	
	q_scale_type = 0	q_scale_type = 1
0	Not allowed	
1	2	1
2	4	2
3	6	3
4	8	4
5	10	5
6	12	6
7	14	7
8	16	8
9	18	10
10	20	12
11	22	14
12	24	16
13	26	18
14	28	20
15	30	22
16	32	24
17	34	28
18	36	32
19	38	36
20	40	40
21	42	44
22	44	48
23	46	52
24	48	56
25	50	64
26	52	72
27	54	80
28	56	88
29	58	96
30	60	104
31	62	112

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Zig-Zag Scanning of DCT Coefficients The zig-zag scan order for DCT coefficients used in MPEG-1 has already been defined in Chapter 4. An alternate zig-zag scan order has been introduced in MPEG-2 and is used when the **alternate_scan** flag is set to one. This alternate zig-zag scan pattern is shown in Figure 6.12.

The alternate scan order scans the coefficients toward the bottom of the first two columns of DCT coefficients considerably earlier in the scan than the other original

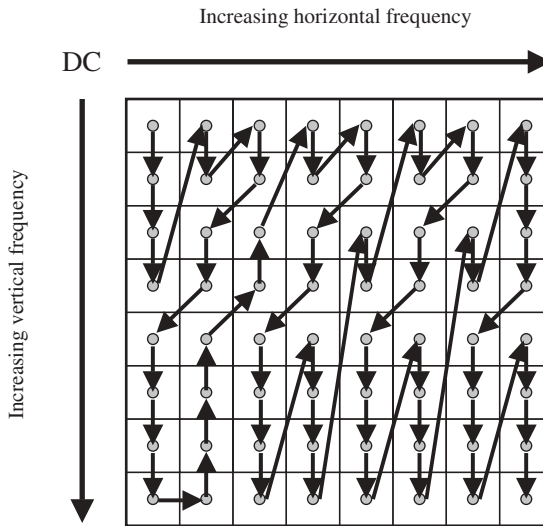


Figure 6.12 Alternate zig-zag scan order.

scan order. It would therefore be suitable for blocks with faster vertical than horizontal changes. This can be expected to occur in field pictures in interlaced sequences, as the 16 vertical pixels in a macroblock effectively span 32 pixels in the equivalent frame picture.

Repeat First Field The 1-bit **repeat_first_field** flag causes the first field of an interlaced frame picture to be repeated. The first field (either top or bottom) is followed by the alternate field, and then the first field is repeated. At the first glance, this might not seem a particularly useful functionality. Its usefulness becomes apparent when we consider the transmission of film material over digital television.

Film material is usually recorded at 24 frames/s. For transmission over a 50-field/s system, the film material is speeded up to 25 frames/s. This results in a barely noticeable change in the pitch of the accompanying audio material. However, for a 60-field/s systems, a process known as 3:2 pulldown is employed. Using this approach, one film frame produces three fields of television information whereas the next film frame produces just two fields. This is illustrated in Figure 6.13.

The first film frame is scanned into three fields, which are then merged to form a single frame for coding and transmission (Fields 1 and 3 can be identical or alternatively averaged to form the top field of Frame 1). For correct display, Frame 1 needs both **repeat_first_field** and **top_field_first** flags set to one. The second film scene is scanned to two fields that are merged to form a single frame for coding and transmission. To keep the required alternate order of fields, which is needed by interlaced displays, the **top_field_first** flag is set to zero in this case as it is the bottom field that needs to be displayed first. The third film frame is treated in identical manner to the first frame, except that the bottom field needs to be displayed first and is the field that is repeated. The fourth frame is treated like the second, except

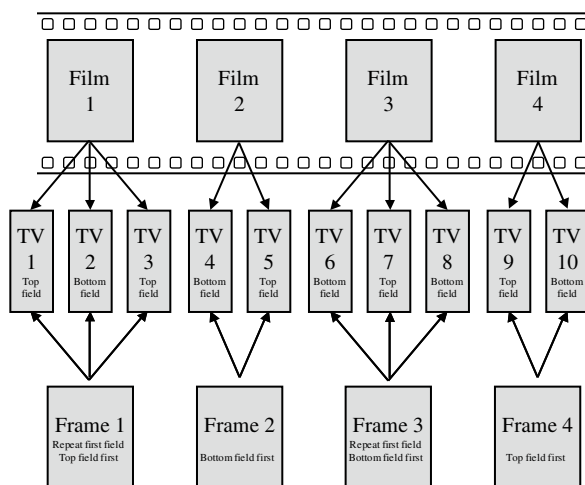


Figure 6.13 Use of 3:2 pulldown in 60-Hz systems.

that the **top_field_first** flag is set to one. This allows a 30-picture/s movie to be displayed while only transmitting 24 pictures/s. This greatly enhances the coding performance for movie material. For field pictures, the **repeat_first_field** flag is always set to zero.

As described in the section on “Top field first”, for progressive sequences the **repeat_first_field** flag is used together with the **top_field_first** flag to indicate the number of times that a decoded progressive frame should be output by the decoder.

Composite Display Flag This flag indicates that the bit stream contains information about the analog composite video prior to encoding. This includes the field sequence, burst amplitude, and subcarrier phase. This information is not needed to correctly decode the digital bit stream but can be useful if the decoded video signal is to be returned to the analog form.

6.3.4.3. Quantizer Matrix Extension

We have already seen at the sequence layer that new intrablock and interblock quantizer matrices can be defined by the user and downloaded in the bit stream. It is possible to redefine these quantizer matrices using the quantizer matrix extension at the picture layer. In addition, the extra flexibility of having different quantizer matrices for luminance and chrominance is also provided at the picture layer for chrominance formats other than 4:2:0. A 1-bit flag indicates whether a particular quantizer matrix is present or not. Thus, up to four quantizer matrices can be loaded in the following order: intraquantizer matrix, interquantizer matrix, chrominance intraquantizer matrix, and chrominance interquantizer matrix.¹⁵

¹⁵ For a 4:2:0 chrominance structure, separate luminance and chrominance quantizer matrices are not available.

When the intraquantizer matrix is loaded, its values are also used for the chrominance intraquantizer matrix, unless this is subsequently defined in the extension. Similarly, when the interquantizer matrix is loaded, its values are also used for the chrominance interquantizer matrix, unless this is subsequently defined in the extension. Similar to the sequence layer, the quantizer matrices are downloaded as sixty-four 8-bit values (range 1–255) in the default zig-zag scan order.

6.3.4.4. Picture Display Extension

The `picture_display_extension()` allows the position of the display rectangle defined in the `sequence_display_extension()` to be moved on a picture-by-picture basis. It begins with a start code followed by the appropriate **extension_start_code_identifier** value as defined in Table 6.3. The position of the display rectangle is defined relative to the center of the complete reconstructed picture as shown in Figure 6.14. The two offsets are 16-bit signed integers giving each offset in units of 1/16th of a pixel.¹⁶ A positive value for the **frame_center_horizontal_offset** indicates that the center of the reconstructed picture lies to the right of the center of the display rectangle. A positive value for the **frame_center_vertical_offset** indicates that the center of the reconstructed picture lies below the center of the display rectangle. In Figure 6.14, both of these offsets take negative values.

The frame center offsets can be used to implement pan and scan so that the display rectangle can be moved around the entire reconstructed picture to ensure that the most “interesting” area is always displayed. As an example, suppose that we have a wide-screen (16×9) image of two people talking at either end of a long table. One person is therefore at the extreme left in the reconstructed picture while the other person is at the extreme right of the picture. For a viewer watching on a standard-definition (4×3) display, looking at the central portion of the reconstructed picture

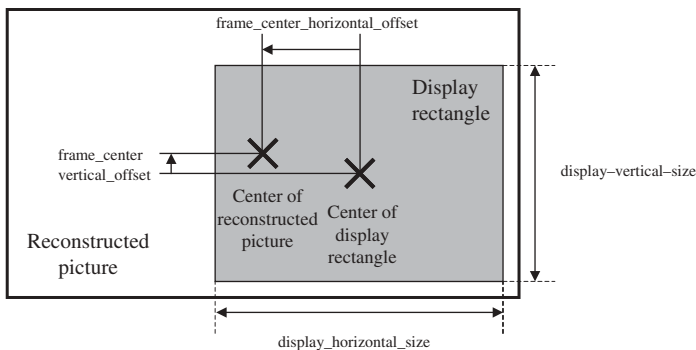


Figure 6.14 Offset of displayed picture relative to reconstructed picture.

¹⁶ Specification to 1/16th of a pixel accuracy is required for scalable applications where higher layers may contain more pixels than lower layers and still require at least pixel accuracy. Scalability is not covered in detail in this text as it is not used in current digital television applications.

would show the table, but possibly neither of the speakers. The use of the frame center offsets allows the displayed area to move both left and right to show each person while they are speaking.

Earlier in this chapter we saw that for interlaced sequences, a field picture always contains exactly one field whereas a frame picture can contain two or three fields depending on the value of the **repeat_first_field** flag. If there is more than one field, then separate values of the **frame_center_horizontal_offset** and the **frame_center_vertical_offset** are required for each displayed field.

Similarly for progressive sequences, the picture can be displayed one, two, or three times depending upon the values of the **repeat_first_field** and **top_field_first** flags. Again, if a picture is displayed more than once then separate values for the two offsets are required for each displayed picture.

6.3.4.5. Copyright Extension

The copyright extension allows materials to be marked as copyright. It again consists of a start code followed by the appropriate **extension_start_code_identifier** value shown in Table 6.3. A 1-bit flag indicates whether the material is copyright or not. When set to one, this indicates that all the coded pictures following the copyright extension up to the next copyright extension or the end of sequence start code are copyright. This is followed by an 8-bit **copyright_identifier**, which indicates the registration authority for the copyright, an **original_or_copy** flag, which indicates that the material is original when set to one and a copy when set to zero, and a 64-bit **copyright_number**, which uniquely identifies the copyright material. To remove the possibility of start-code emulation (i.e., 23 or more successive zero bits), the copyright number is transmitted as a 20-bit field and two 22-bit fields with each field separated by a marker bit, which is set to one.

6.3.4.6. User Data

User data can also be included at the picture layer. It begins with the unique **user_data_start_code** and has a format identical to that used for user data at the sequence layer.

6.3.5. The Slice Layer

A *slice* consists of a number of consecutive macroblocks all in the same row of macroblocks. A slice can contain all of the macroblocks in a particular row of macroblocks in a picture or a series of macroblocks in the same row of macroblocks. This is shown in Figure 6.15. Each row of macroblocks starts with a new slice header. The first row of macroblocks in Figure 6.15 forms a single slice while the second row of macroblocks forms two slices. There is no limit to the number of macroblocks that can be included in a slice, except that each slice must contain at least one macroblock and all macroblocks in a slice must be in the same row of macroblocks. The first and last macroblocks in a slice cannot be skipped macroblocks.

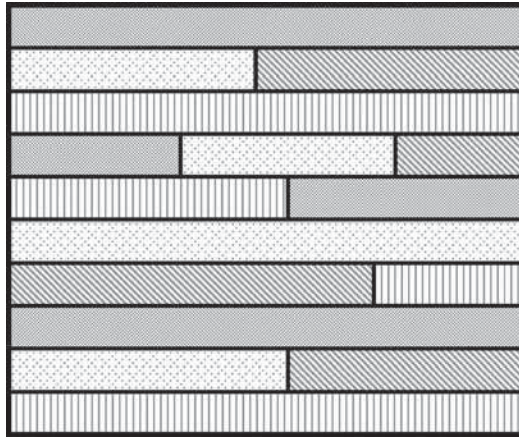


Figure 6.15 Arrangement of slices in a picture (restricted slice structure).

Requiring all macroblocks in a picture to be covered by a slice is called the *restricted slice structure*. This structure is used in all current digital television applications. The *unrestricted slice structure* allows gaps of unencoded macroblocks between slices as shown in Figure 6.16. In this case, it is clear that the first and last macroblocks in each slice need to be coded, so that the parts of the picture coded can be uniquely defined.

Each slice begins with a slice start code that consists of the start code 0x000001 followed by an 8-bit **slice_vertical_position**, which takes values in the range 0x01–0xAF (in decimal 1–175). The **slice_vertical_position** defines which row of macroblocks is contained in the slice with the first row of macroblocks in a picture numbered one, the second row of macroblocks numbered two, and so on. If a row of macroblocks is made up of more than one slice, then each of these slices has the same value for **slice_vertical_position**. The horizontal position of the first

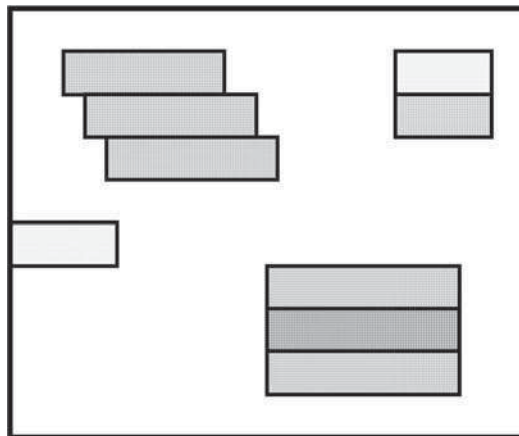


Figure 6.16 Arrangement of slices in a picture (unrestricted slice structure).

macroblock in each slice is defined in the macroblock header as is discussed in the section on ‘Macroblock address’.

6.3.5.1. Slice Number Extension for Pictures With More Than 2800 Lines

A picture with more than 2800 lines of pixels (i.e., more than 175 rows of macroblocks) requires more slices than are allowed by the range of **slice_vertical_position**. For pictures of this size (as defined at the sequence layer), a further 3-bits immediately follows the **slice_start_code**. This field is called the **slice_vertical_position_extension** and extends the range of slice numbers from 1 to 1023 by acting as the three MSBs of the **slice_vertical_position**. This range is sufficient for all picture sizes permitted by the MPEG-2 specification.

6.3.5.2. Quantizer Step Size

Next follows the 5-bit **quantizer_scale_code** field that indicates the quantizer to be used for macroblocks in the slice. The mapping between **quantizer_scale_code** and quantizer value is given in Table 6.12 for each value of **q_scale_type** (defined at the picture layer). The quantizer value can be redefined in the macroblock header of any macroblock that is not skipped. In practice, it is not usually desirable to do this for every macroblock because of the large overhead involved.

6.3.5.3. Intraslice

The **intra_slice_flag** field when set to one indicates the presence in the bit stream of an **intra_slice** flag as well as further **reserved_bits** and extra slice information. The **intra_slice** flag, when set to one, indicates that all macroblocks in the slice are coded in intramode. This can be useful in allowing a decoder to perform fast forward and fast reverse operations. The **intra_slice** flag is followed by a 7-bit field (**reserved_bits**), which is reserved for future use and must currently take the value zero. The extra slice information consists of an **extra_bit_slice** flag, which if set to one, indicates that it is followed by an 8-bit **extra_information_slice** field. Then follows another **extra_bit_slice** flag, and the process continues until the **extra_bit_slice** flag takes the value zero. In the current systems, the **extra_bit_slice** flag should be set to zero and any information following the **extra_bit_slice** flag, if set to one, should be ignored by the decoder.

The slice header is followed by a series of coded macroblocks of data until a further start code is encountered.

6.3.6. The Macroblock Layer

The macroblock header is used to define the address of the macroblock, the macroblock type, the quantizer to be used, the motion vector(s), and the coded block pattern. We consider each of the elements one by one. Not all of these elements are

```

macroblock_escape

macroblock_escape

macroblock_address_increment = 22

```

Figure 6.17 Representation of a **macroblock_address_increment** of 88.

present in every macroblock header. However, the decoder can correctly determine which elements to expect from the macroblock-type information.

6.3.6.1. Macroblock Header

Macroblock Address The macroblock address defines the horizontal position of the macroblock within a row of macroblocks. The row of macroblocks in which the macroblock is located can be inferred from the slice header. As explained in Chapter 5, the data transmitted is actually the difference between the address of the current macroblock and the address of the last transmitted macroblock and is called the **macroblock_address_increment**. A variable-length code is used to represent these differences with a range of 1–33. As discussed when considering the sequence layer, MPEG-2 can deal with very large pictures. It is therefore possible that the **macroblock_address_increment** might need to be larger than 33. In this case, it is preceded by one or more fixed-length 11-bit code words called the **macroblock_escape**. Each time the **macroblock_escape** is included in the bit stream, the **macroblock_address_increment** is increased by 33. Thus, if the difference in address between the last-coded and the current macroblocks was 88 macroblocks, then this would be encoded as shown in Figure 6.17.

After the slice header, the address of the first macroblock is coded as its actual value rather than the difference from the last-encoded macroblock in the previous slice. This allows the decoder to correctly determine the position of this macroblock even if transmission errors have made it impossible for the previous slice to be correctly decoded. The slice header gives the vertical position of the macroblock while the macroblock header gives the horizontal position. All the coded macroblocks contain a **macroblock_address_increment**.

Macroblock Type The macroblock-type information depends on the type of picture (I, P, or B) that is currently being decoded. We consider each picture type separately.

I Pictures All macroblocks in I pictures are coded in intramode. As shown in Table 6.13, there are only two types of macroblocks in I pictures, namely macroblocks where the quantizer step size is redefined and macroblocks where the quantizer remains unchanged. Motion vectors are not needed for the decoding of I macroblocks.¹⁷ Since every block in an I macroblock needs to be coded as it must contain

¹⁷ Other than concealment motion vectors if specified at the picture layer. All intramacroblocks in a picture that uses concealment motion vectors contain a motion vector and so this does not need to be determined at the macroblock layer.

Table 6.13 Macroblock types for I pictures.

Macroblock-type VLC code word						
New quantizer defined						
forward motion vector						
Backward motion vector						
Coded block pattern						
Intramacroblock						
Description of macroblock						
1	No	No	No	No	Yes	Intra, no new quantizer
01	Yes	No	No	No	Yes	Intra, new quantizer

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

at least a nonzero DC DCT coefficient, there is no need for a coded block pattern to indicate which blocks are coded.

P Pictures The types of macroblocks that can occur in a P picture are shown in Table 6.14.

Table 6.14 Macroblock types for P pictures.

Macroblock-type VLC code word						
New quantizer defined						
forward motion vector						
Backward motion vector						
Coded block pattern						
Intra macroblock						
Description of macroblock						
1	No	Yes	No	Yes	No	MC, coded blocks, old quant
01	No	No	No	Yes	No	Inter, coded blocks, old quant
001	No	Yes	No	No	No	MC, no coded blocks, old quant
00011	No	No	No	No	Yes	Intra, old quant
00010	Yes	Yes	No	Yes	No	MC, coded blocks, new quant
00001	Yes	No	No	Yes	No	Inter, coded blocks, new quant
000001	Yes	No	No	No	Yes	Intra, new quant

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

The macroblock types can be divided into three groups: macroblocks that utilize forward motion-compensated prediction from the previous I or P reference picture (motion-compensated mode), macroblocks that use the macroblock in the same position in the previous I or P reference picture for prediction (intermode), and macroblocks that do not use any form of temporal prediction (intra mode).

For macroblocks coded using motion-compensated prediction, the possibilities are as follows:

- One or more blocks contains coded data and the quantizer is unchanged.
- No block contains coded data and the quantizer is unchanged. The macroblock still needs to be transmitted to send the appropriate motion vector.
- One or more blocks contain coded data and a new quantizer value is used.

There is no need to define a macroblock type for the case where no block contains coded data and a new quantizer value is used, because if no blocks contain coded data then all reconstructed blocks are zero irrespective of the quantizer value used.

For macroblocks coded using interprediction, there are two possibilities

- One or more blocks contain coded data and the quantizer is unchanged.
- One or more blocks contain coded data and a new quantizer value is used.

An intercoded macroblock in which no block has a coded data is a skipped macroblock. The reconstructed value of a skipped intermacroblock (all zeros) is the same irrespective of the quantizer. As with motion-compensated macroblocks, there is therefore no need to define an intermacroblock that only defines a new quantizer value but for which all blocks of data are zero.

The case for intracoded macroblocks is the same as that for I pictures, that is, the macroblocks are coded using the current quantizer or have a new quantizer value defined in the macroblock header. Intramacroblocks are always coded.

B Pictures The types of macroblocks that can occur in a B picture are shown in Table 6.15.

In this case, there are four different classes of macroblocks. Interpolated macroblocks require two motion vectors: a forward motion vector that points to a previous I or P reference picture and a backward motion vector that points to a future I or P reference picture (interpolated prediction). Two motion vectors are transmitted in the macroblock header. The final prediction is the average of the two predictions generated. The result of motion-compensated prediction and coding can result in all blocks being zero. In this case, the quantizer is assumed to be unchanged because decoding produces the same result irrespective of the quantizer used. Alternatively, one or more blocks might contain the coded data in which case the quantizer can remain the same or be redefined in the macroblock header.

The same three macroblock types are used for macroblocks coded with forward motion-compensated prediction only (forward prediction) and for macroblocks coded with backward motion-compensated prediction only (backward prediction).

Table 6.15 Macroblock types for B pictures.

Macroblock-type VLC code word						
	New quantizer defined					
		forward motion vector				
			Backward motion vector			
				Coded block pattern		
					Intramacroblock	
					Description of macroblock	
10	No	Yes	Yes	No	No	Interp, no coded blocks, old quant
11	No	Yes	Yes	Yes	No	Interp, coded blocks, old quant
010	No	No	Yes	No	No	Back, no coded blocks, old quant
011	No	No	Yes	Yes	No	Back, coded blocks, old quant
0010	No	Yes	No	No	No	Fwd, no coded blocks, old quant
0011	No	Yes	No	Yes	No	Fwd, coded blocks, old quant
00011	No	No	No	No	Yes	Intra, old quant
00010	Yes	Yes	Yes	Yes	No	Interp, coded blocks, new quant
000011	Yes	Yes	No	Yes	No	Fwd, coded blocks, new quant
000010	Yes	No	Yes	Yes	No	Back, coded blocks, new quant
000001	Yes	No	No	No	Yes	Intra, new quant

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

In each of these cases, only one motion vector is transmitted in the macroblock header.

Finally, there can still be intracoded macroblocks in B pictures. As for I and P pictures, an intracoded macroblock can use the current quantizer or have a new quantizer defined in the macroblock header. Intramacroblocks in B pictures are always coded.

In B pictures, a skipped macroblock is a macroblock that uses exactly the same form of prediction with exactly the same motion vectors as the macroblock that immediately precedes it. For this reason, the macroblock immediately after an intracoded macroblock cannot be a skipped macroblock in a B picture.

Types of Motion-Compensated Prediction For macroblocks that are coded using motion-compensated prediction, the next field in the bit stream specifies the prediction type, unless this has been already determined for a frame picture by the fact that the **frame_pred_frame_dct** flag has been set to one at the picture layer. The field specifying the prediction type is the **frame_motion_type** for frame pictures or the **field_motion_type** for field pictures. Each of these 2-bit fields defines the type of motion compensation used to code the macroblock.

Table 6.16 Definition of **frame_motion_type**.

frame_motion_type	Prediction type	Number of motion vectors	Motion vector format	Differential motion vector
00	Reserved	—	—	—
01	Field	2	Field	No
10	Frame	1	Frame	No
11	Dual prime	1	Field	Yes

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Motion-Compensated Prediction for Frame Pictures The type of motion-compensated prediction used in frame pictures is specified by the **frame_motion_type** field and defined in Table 6.16. There are three possible forms of motion-compensated prediction: field prediction, frame prediction, and dual-prime prediction.

The simplest form of prediction is the frame prediction where a macroblock in the current frame picture is predicted by the best matching macroblock in a previous I or P frame picture (P pictures) or from a previous and/or a future I or P frame picture (B pictures). The frame prediction process for a P picture is illustrated in Figure 6.18¹⁸ for a pair of vertically adjacent macroblocks. For both current and reference pictures, the two fields are merged to form a single frame reference picture. This reference frame picture may originally have been transmitted as two field pictures. A macroblock in the current frame picture is then predicted from the reference picture. Thus, in the top macroblock shown in Figure 6.18, the top field pixels in the current picture are predicted from top field pixels in the reference picture whereas in the bottom macroblock top field pixels in the current frame picture are predicted from bottom field pixels in the reference picture. The vertical component of the motion vector relative to a pixel in the current picture (shown surrounded by a square) is also shown. Thus, in the top macroblock, the vertical component of the motion vector is +2 whereas in the bottom macroblock it is -5.

The situation is similar for frame prediction in B pictures and is illustrated in Figure 6.19. Once again the fields of the current picture and the two reference pictures are merged to form frame pictures. One or both of the reference frame pictures may originally have been transmitted as field pictures. In the top macroblock shown in Figure 6.19, interpolative prediction is employed. Thus, both forward and backward motion vectors are used. In the bottom macroblock, only backward prediction is employed, and so only a single motion vector is required. The values of the vertical component of the motion vectors are again shown relative to a pixel in the current picture (shown surrounded by a square).

As shown in Table 6.16, only a single motion vector is required for frame prediction of a P picture. Since prediction is required in up to two directions for interpolated prediction in a B picture, up to two motion vectors may be required.

¹⁸ All of the diagrams in this section of the text show motion vectors with integer components. This is for clarity. In reality, motion-compensated prediction is performed to half-pixel accuracy in MPEG-2.

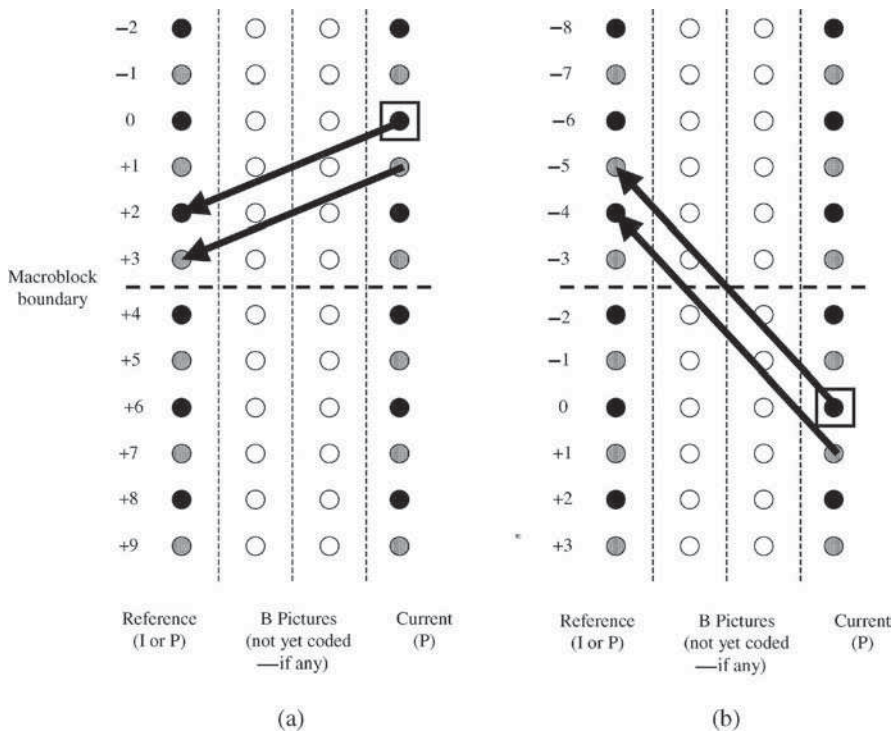


Figure 6.18 Frame prediction of a P frame picture: (a) top macroblock and (b) Bottom macroblock.

Now let us consider field prediction in frame pictures. In this case, the pixels in a macroblock are split according to whether they come from the top field or the bottom field. A separate motion vector is then used to encode each field. The reference fields used for prediction are the most recently decoded reference top field and the most recently decoded reference bottom field prior to the current frame picture. Each of the fields in the frame picture can be predicted from either of the reference fields. There is no requirement for the top field to be predicted from the top reference field, the bottom field from the bottom reference field, or even each field from a different reference field.

Figure 6.20 shows the situation for field prediction in a P frame picture. In the top macroblock, both fields are predicted from the bottom reference field. In the bottom macroblock, the top field is again predicted from the bottom reference field whereas the bottom field is predicted from the top reference field. The vertical motion vector displacements are again shown in the diagram and are identical for the two reference pixels in the two fields of the current frame picture (indicated by a square). Thus, a vertical displacement of zero when the top field is predicted from the top reference field refers to the pixel on the same line as the pixel in the current picture. A vertical displacement of zero when the top field is predicted from the bottom reference field is actually predicted from the line in the frame picture directly below (and therefore in the other field from) the line containing the current pixel.

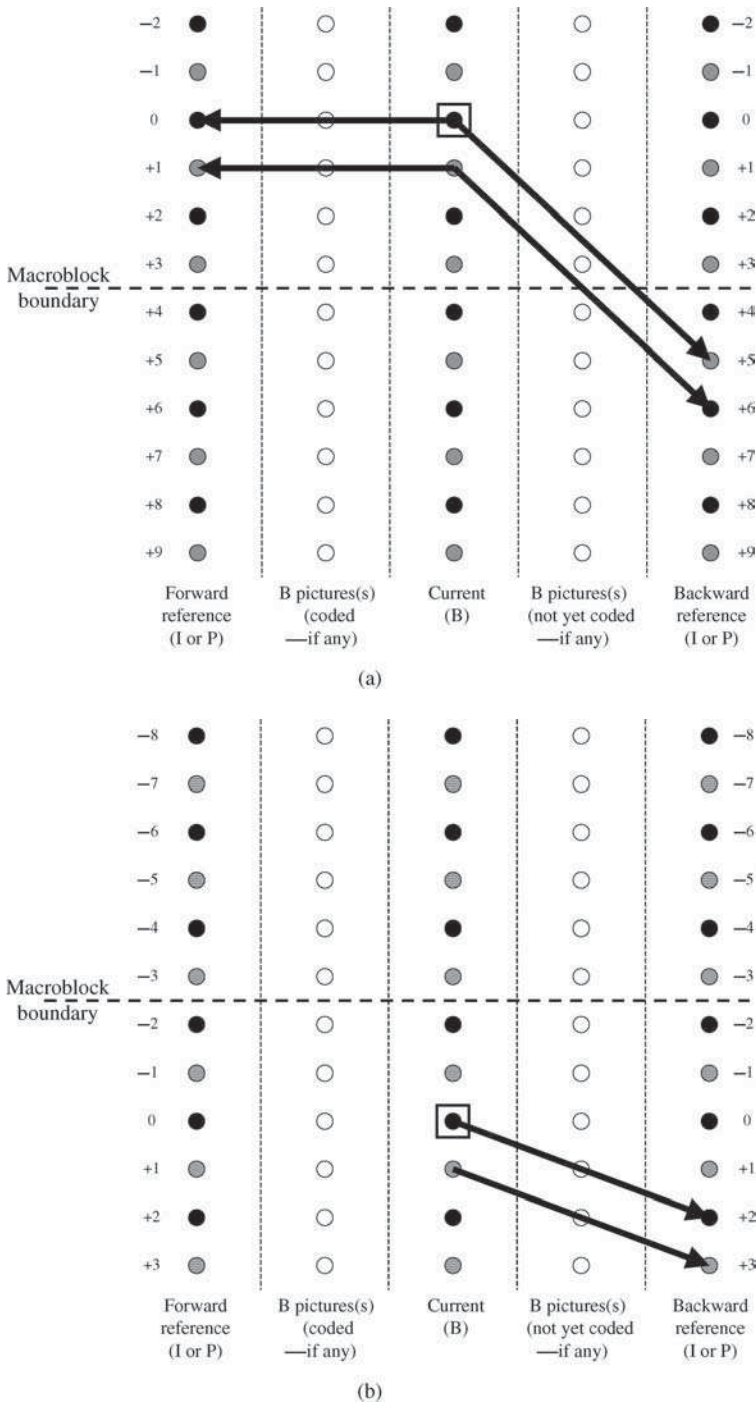


Figure 6.19 Frame prediction of a B frame picture: (a) top macroblock and (b) bottom macroblock.

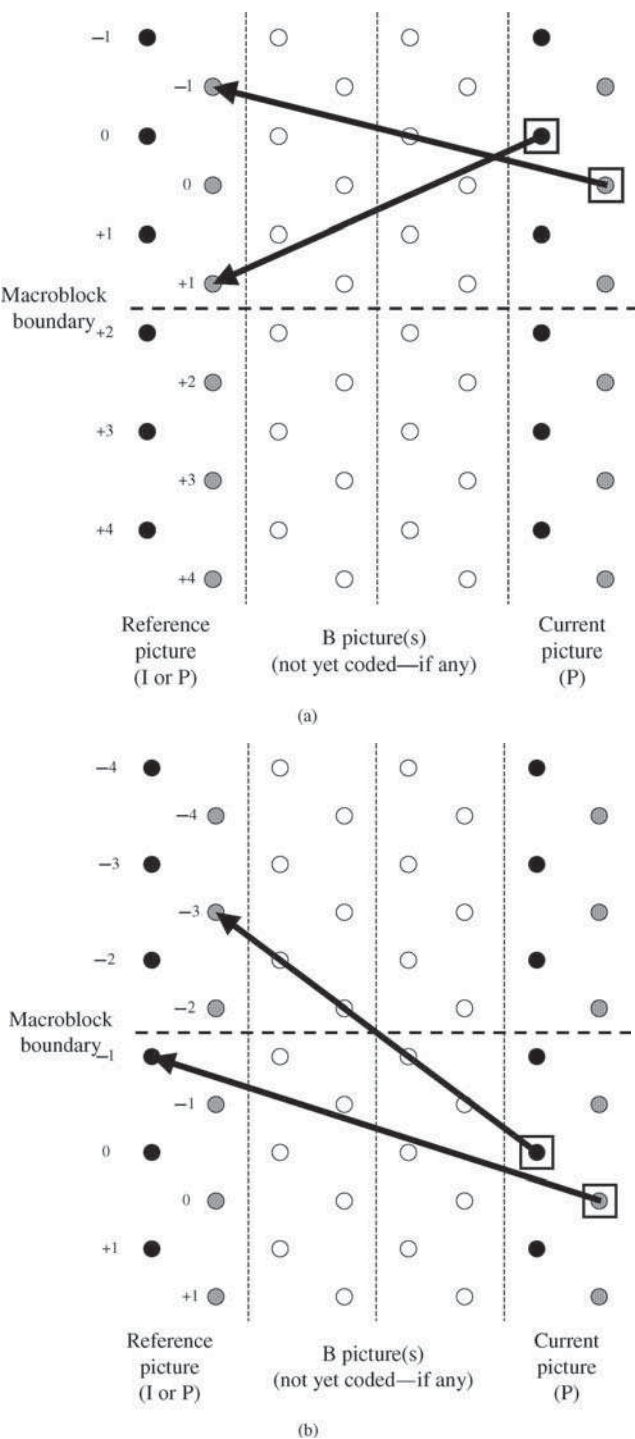


Figure 6.20 Field prediction in a P frame picture: (a) top macroblock and (b) bottom macroblock.

A similar situation applies when the bottom field is being predicted, except that in the case of prediction from the top field, a vertical displacement of zero refers to the pixel in the line directly above (and therefore in the other field from) the line containing the current pixel. In all cases, a zero vertical displacement refers to the same line number in the field as the line number in the field of the current picture.

Comparing Figure 6.20 with Figure 6.18, we can see that a vertical displacement of +1 using field prediction is the same as a vertical displacement of +2 using frame prediction. This is the reason, as shown in the section on “Motion vector range”, that the vertical motion vector range used for field prediction is half that for frame prediction as well as half that for the horizontal motion vector range in both modes.

The situation for the use of field prediction in a B frame picture is shown in Figure 6.21. In this case, the top macroblock is coded using interpolative prediction. As shown, a total of four motion vectors are required. In the bottom macroblock, only backward prediction is employed, and so only two motion vectors are required in this case.

In summary, field prediction in frame pictures requires two motion vectors for macroblocks in P pictures and two motion vectors (backward prediction only or forward prediction only) or four motion vectors (interpolative prediction) for B pictures. In the latter case, an encoder needs to carefully balance the additional overhead associated with transmitting additional motion vectors with the saving in bits required to represent the prediction difference.

The final form of motion-compensated prediction used in frame pictures is dual-prime motion-compensated prediction. This is a special motion-compensated prediction technique developed during the MPEG-2 standardization process. Dual-prime motion-compensated prediction can only be used for P pictures and only when the current picture and the reference picture from which prediction is to occur are concurrent in display order (i.e., not separated by any intervening B pictures).

Dual prime operates by sending a single full motion vector and a small differential motion vector. Each component of the differential motion vector can only take one of the three values: -0.5 , 0 , or $+0.5$. The prediction mode for a frame picture is shown in Figure 6.22.

Each field within the macroblock to be predicted is treated separately. A single motion vector that is read from the bit stream is used to perform field prediction for each field with the top field being predicted from the top field of the reference picture and the bottom field being predicted from the bottom field of the reference picture. This motion is then interpolated or extrapolated to make a prediction from the other field in the reference frame picture. In the case of the top field shown in Figure 6.22, the motion vector used to form a prediction from the bottom field in the reference frame picture is half the motion vector used to form a prediction from the top reference field since the temporal distance is halved. Similarly, for the bottom field shown in Figure 6.22, the motion vector used to form a prediction from the top reference field is one and a half times the motion vector used to form a prediction from the bottom reference field since the temporal distance is one and a half times as great.

Where a component of the field motion vector is not an integer, the motion vector calculated by extrapolation or interpolation contains a motion vector to quarter-pixel

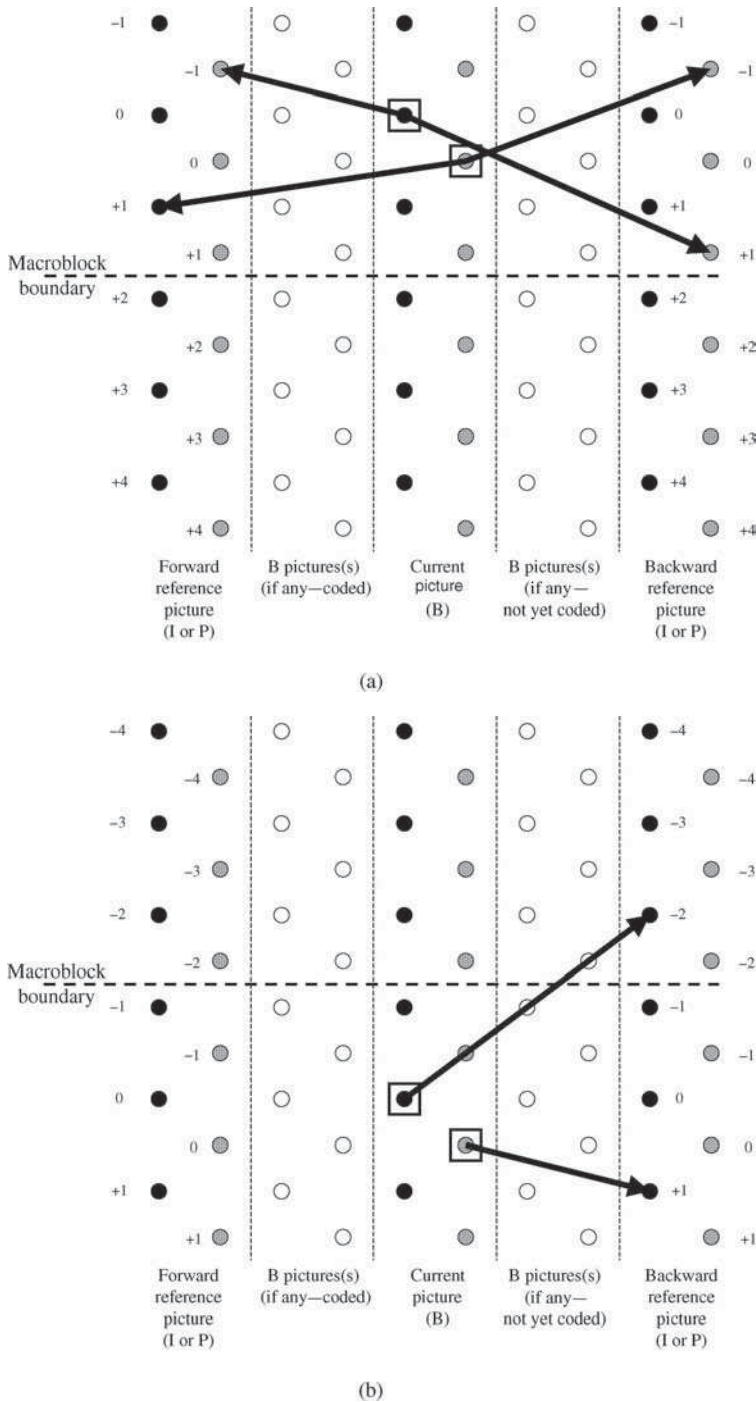


Figure 6.21 Field prediction in a B frame picture: (a) top macroblock and (b) bottom macroblock.

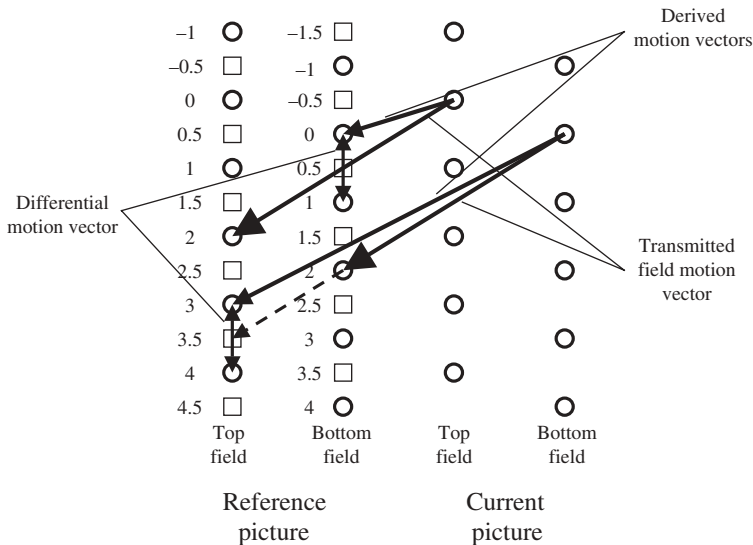


Figure 6.22 Dual-prime motion-compensated prediction for frame pictures.

accuracy. For example, a field motion vector of $(+2.5, -3.5)$ would be interpolated to $(+1.25, -1.75)$ and extrapolated to $(+3.75, -5.25)$. Rounding is performed to the nearest half-pixel value away from zero. Thus, in the case just considered, the interpolated motion vector would be $(+1.5, -2.0)$ and the extrapolated motion vector $(+4.0, -5.5)$. As shown in Figure 6.22, for the vertical component of the motion vector, a 0.5-pixel offset needs to be included to account for the vertical displacement between top and bottom fields.

The motion vectors calculated by extrapolation and interpolation are further modified by the differential motion vector, each component of which is limited to the three values: -0.5 , 0 , or $+0.5$. The two predictions for each field within the macroblock are then averaged to form the final prediction. The following example illustrates the operation of dual-prime motion-compensated prediction.

EXAMPLE 6.1

The field motion vector for dual-prime motion-compensated prediction in a frame picture is $(+2, -2)$. The differential motion vector is $(-0.5, +0.5)$. Calculate the displacements of the information to be used to predict each field in the macroblock. Assume that the top field is transmitted first for each picture.

We consider the prediction of the top and bottom fields within the macroblock separately.

Top Field

Predicted from top field of reference picture $(+2, -2)$

Predicted from bottom field of reference picture

Vertical component = Half vertical component of field motion vector + Field offset (− 0.5 in this case) + Vertical component differential motion vector
= $0.5 \times (+2) - 0.5 - 0.5 = 0$

Horizontal component = Half horizontal component of field motion vector + Horizontal component differential motion vector
= $0.5 \times (-2) + 0.5 = -0.5$

Bottom Field

Predicted from bottom field of reference picture (+2, −2)

Predicted from top field of reference picture

Vertical component = 1.5 times vertical component of field motion vector + Field offset (+0.5 in this case) + Vertical component differential motion vector
= $1.5 \times (+2) + 0.5 - 0.5 = 3$

Horizontal component = 1.5 times horizontal component of field motion vector + Horizontal component differential motion vector
= $1.5 \times (-2) + 0.5 = -2.5$ ■

Motion-Compensated Prediction for Field Pictures The type of motion-compensated prediction used in field pictures is specified by the **field_motion_type** field and defined in Table 6.17. There are three possible forms of motion-compensated prediction: field prediction, 16 × 8 prediction, and dual-prime prediction.

The simplest of these is field prediction, where the macroblock in the current field picture is predicted from a displaced macroblock in a previous I or P field picture (P pictures) or a previous and/or a future I or P field picture (B pictures). The reference pictures used are the most recently decoded I or P field pictures. These reference field pictures may themselves have been coded as frame pictures. The situation for the first P field picture (which is assumed to be the top field) is shown in Figure 6.23. In the top macroblock, prediction is made from the previous top reference field whereas in the bottom macroblock prediction is made from the bottom reference field.

The situation for the second P field picture is shown in Figure 6.24. Remember that prediction is performed from the two most recently received reference fields. This means that one of the reference fields is the P field picture just received and decoded together with the last received reference field of the type opposite (top or bottom) to the P field picture just decoded. In Figure 6.24, the top macroblock is

Table 6.17 Definition of **field_motion_type**.

field_motion_type	Prediction type	Number of motion vectors	Motion vector format	Differential motion vector
00	Reserved	—	—	—
01	Field	1	Field	No
10	16 × 8 MC	2	Field	No
11	Dual prime	1	Field	Yes

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

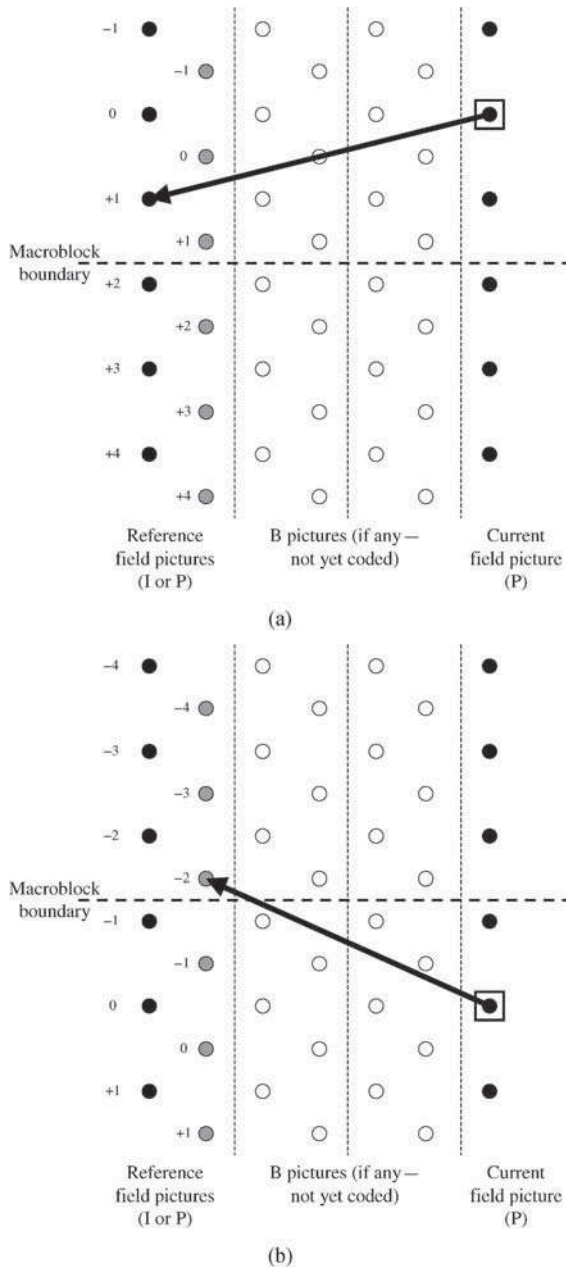


Figure 6.23 Field prediction of the first P field picture: (a) top macroblock and (b) bottom macroblock.

predicted from the P top field picture just received whereas the bottom macroblock is predicted from the bottom reference field received some time earlier.

The situation for the prediction of a B field picture is shown in Figure 6.25. Interpolative motion-compensated prediction is used for the top macroblock requiring

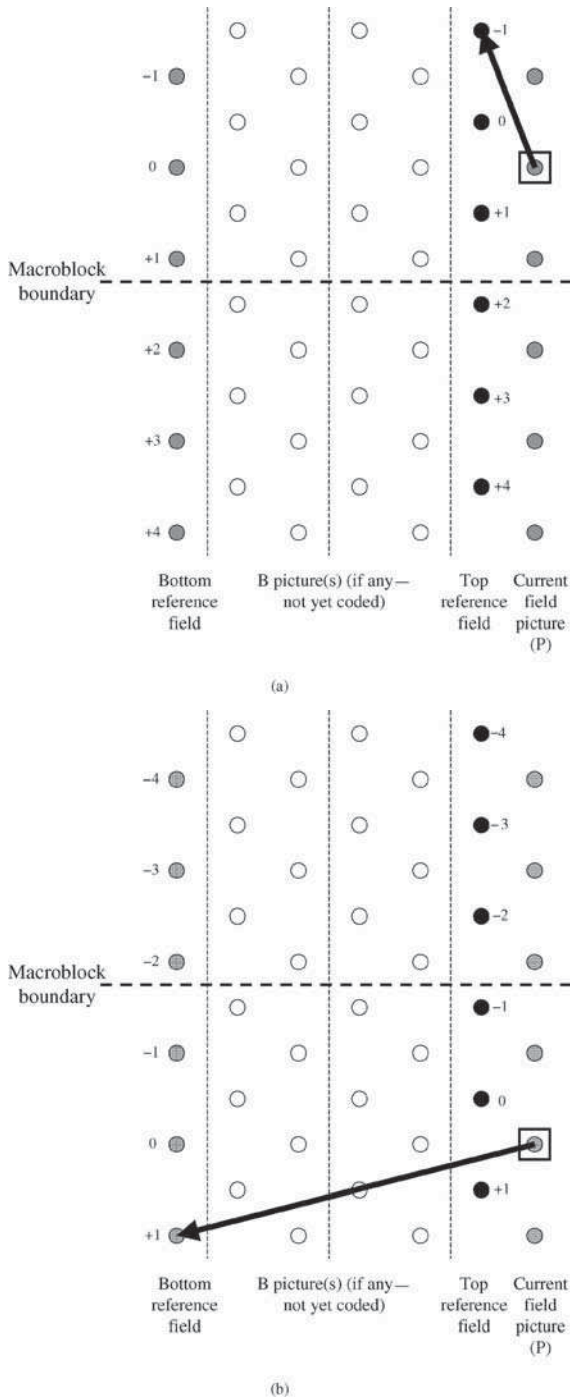


Figure 6.24 Field prediction of the second P field picture: (a) top macroblock and (b) bottom macroblock.

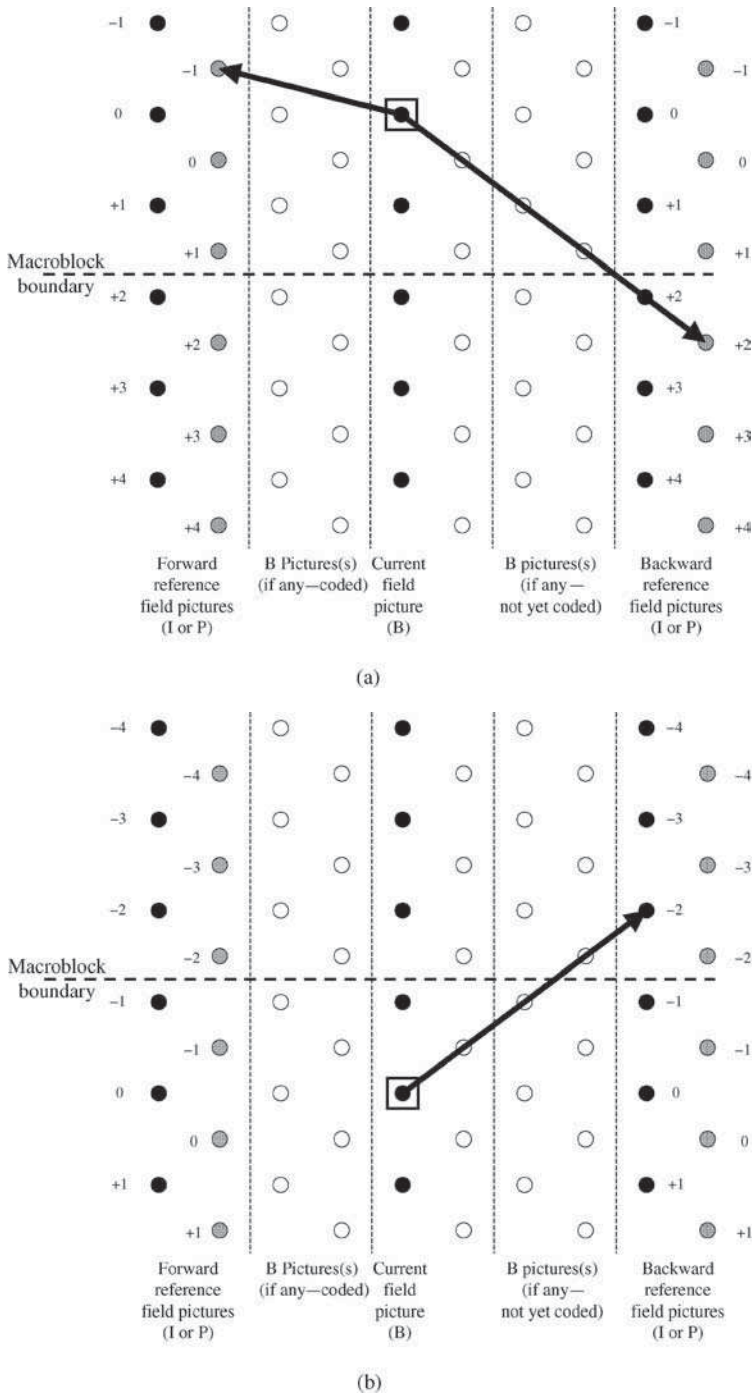


Figure 6.25 Field prediction for the first B field picture: (a) top macroblock and (b) bottom macroblock.

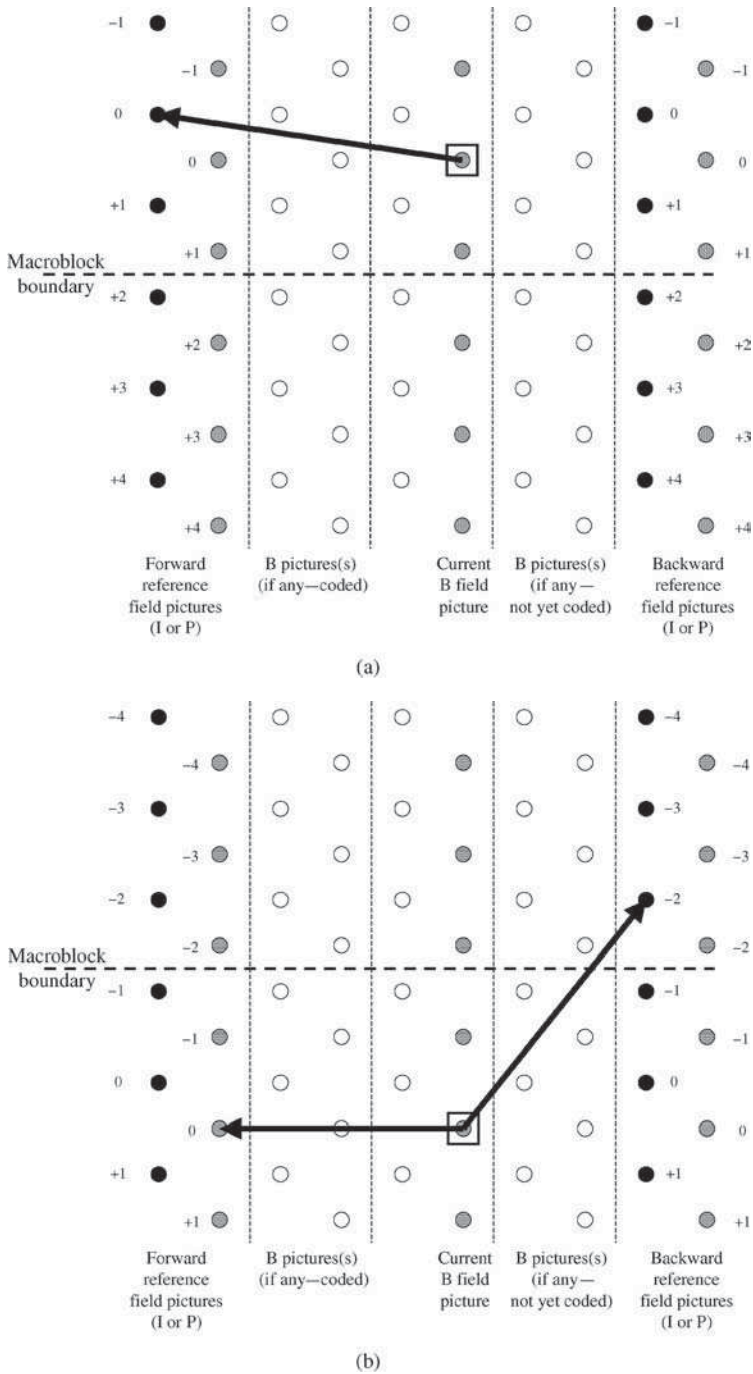


Figure 6.26 Field prediction for a second B field picture: (a) top macroblock and (b) bottom macroblock.

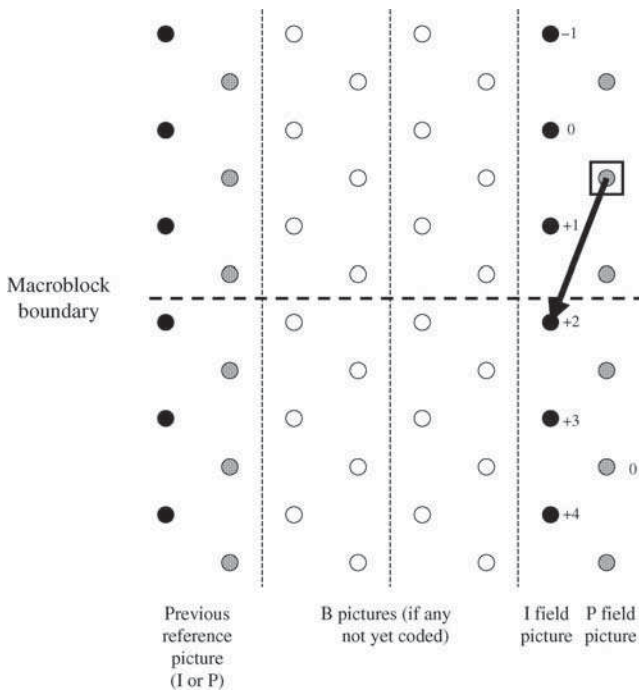


Figure 6.27 Second of a pair of I field pictures coded as a P field picture.

two motion vectors. Only backward prediction is used for the bottom macroblock, and so only a single motion vector is required. The case for the second B picture is shown in Figure 6.26. Unlike P pictures, the appropriate reference pictures for the second B picture are identical to those for the first, and so the prediction process is also identical.

There is another special type of prediction possible in field pictures. This occurs in the case of a pair of field pictures that make up an I frame picture. In this case, the second field picture can be coded as a P field picture. However, all predictions must be from the immediately preceding I field picture as shown in Figure 6.27.

As shown in Table 6.17, only a single motion vector is required for field prediction for macroblocks in P field pictures. For B field pictures, either one motion vector (forward prediction or backward prediction) or two motion vectors (interpolative prediction) are required.

Figure 6.28 shows the top field of the first picture of the “Mobile and Calendar” sequence. When compared to the full frame picture, it is clear that the picture changes twice as quickly vertically in a field picture as in the equivalent frame picture (i.e., the entire image is contained in half as many lines). This means that the chance of more than one type of motion within a macroblock (which is still 16×16 luminance pixels in a field picture) is increased. For this reason, a new motion compensated prediction mode has been introduced for field pictures and is called 16×8 motion compensation. In this mode, two motion vectors (four in the case of a macroblock in a B picture



Figure 6.28 Top field of the first picture of the mobile and calendar sequence.

where both forward and backward predictions are employed) are included in the macroblock. The first motion vector (pair of motion vectors) is used for the prediction of the top 16×8 pixels in the macroblock whereas the second motion vector (pair of motion vectors) is used for the prediction of the bottom 16×8 pixels in the macroblock.

Finally, dual-prime prediction is also available for field pictures. As shown in Figure 6.29, the field motion vector is used to form a prediction from the reference field of the same parity (top or bottom) as the current field picture. A further prediction is formed from the reference field of opposite parity in exactly the same manner as that for dual-prime prediction for frame pictures.

Motion Vector Prediction As was the case in the syntax described in Chapter 5, the values of the transmitted motion vectors in MPEG-2 are predicted using the values of the previously transmitted motion vectors. This prediction is set to zero after a slice header. This allows correct decoding of a new slice even if transmission errors

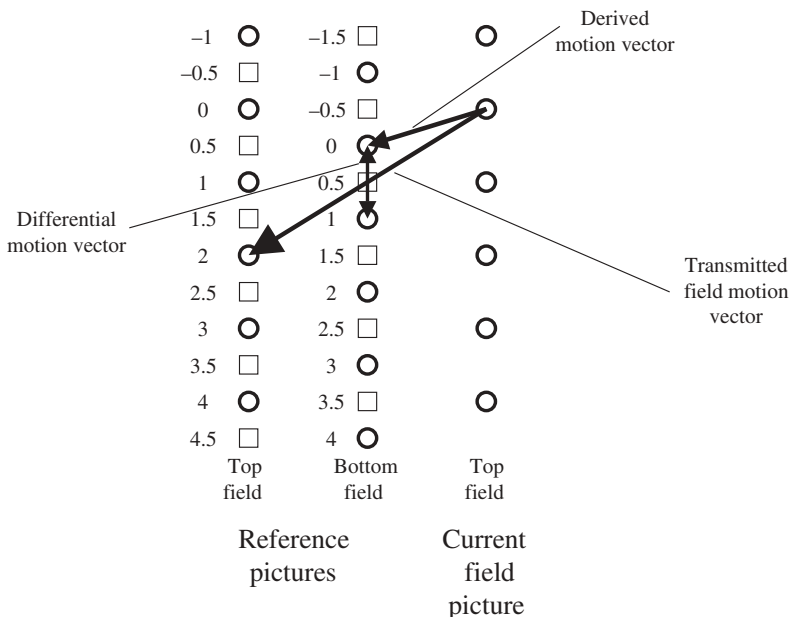


Figure 6.29 Dual-prime motion-compensated prediction for a top field picture.

prevented the correct decoding of the previous slice. For P pictures, the motion vector in the current macroblock is predicted from the motion vector in the previous macroblock, if it made use of motion-compensated prediction (i.e., was not coded as either an intramacroblock or an intermacroblock). Otherwise a prediction of zero is used. This implies that the motion vector predictor is set to zero after a skipped macroblock in a P picture.

For B pictures, there needs to be separate predictors for forward and backward motion vectors. In each case, the predictor is the most recently transmitted motion vector of that type in the current slice, if any. A skipped macroblock in a B picture does not set to zero the predictor because a skipped macroblock in a B picture is assumed to use the same form of motion-compensated prediction as the macroblock that came immediately before it.

This has given a brief introduction of the way that motion vectors are coded. There is considerably more detailed information contained in the standards documentation.

Type of DCT The `dct_type` flag indicates whether a macroblock in a frame picture uses frame DCT coding or field DCT coding. The difference between these two modes is shown in Figure 6.30.

In frame-based DCT, the 16×16 pixel macroblock is simply split into four 8×8 pixel blocks prior to transformation by the DCT. As shown, each block still contains pixels from both fields. In field-based DCT, the lines in the macroblock are rearranged so that all the lines from one field are contained in the top eight lines of the macroblock with the eight lines from the other field contained in the bottom eight lines. The rearranged macroblock is then split into 8×8 pixel blocks and then transformed by the DCT. As shown in Figure 6.30, this rearrangement ensures that all pixels within a block are from the same field.

Field-based DCT is useful for blocks where movement has occurred between fields. Figure 6.31 shows a simple macroblock containing an 8×8 pixel black block

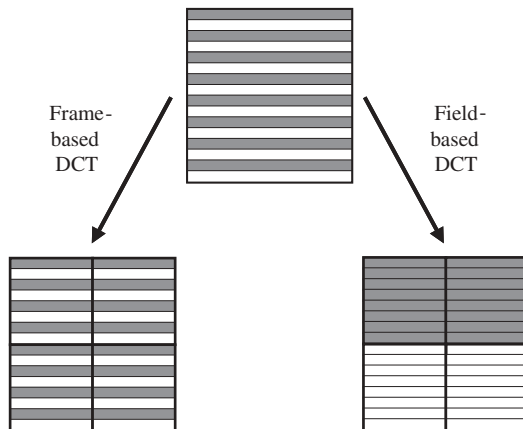


Figure 6.30 Frame-based and field-based DCT coding.

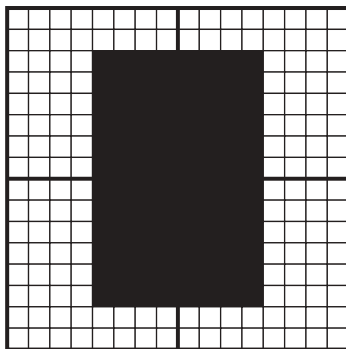


Figure 6.31 Example macroblock.

on a white background. If we assume that the block starts to move to the right at the rate of 2 pixels/field, then after the first field of motion, the macroblock would look as shown in Figure 6.32.

Looking at the pixels in columns 5, 6, 13, and 14, we observe high-frequency vertical changes of intensity. After the application of the DCT, considerable energy appears near the bottom left of the block of DCT coefficients. This leads to inefficient coding as a large number of bits are required to represent DCT coefficients in this area.

The macroblock after rearrangement for field-based DCT coding is shown in Figure 6.33. The high-frequency vertical changes are no longer present, and so improved coding efficiency after the DCT can be expected.

Coded Block Pattern The macroblock header can also contain a coded block pattern to indicate which blocks within the macroblock are coded where this needs to be specified. For digital television, which uses the 4:2:0 color sampling structure, this is identical to the use of the coded block pattern as described in Chapter 5.

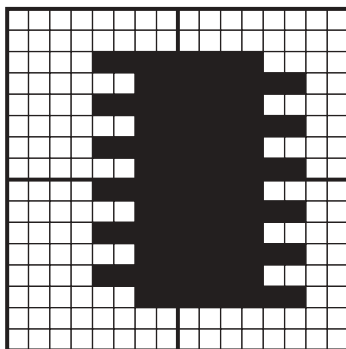


Figure 6.32 Object shown in Figure 6.31 with motion of 2 pixels/field.

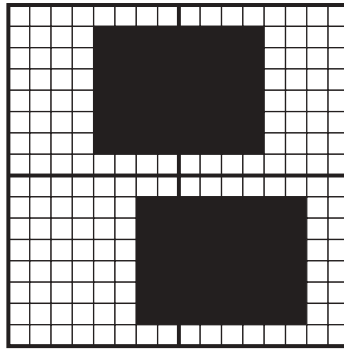


Figure 6.33 Macroblock shown in Figure 6.32 after rearrangement for field DCT coding.

6.3.7. The Block Layer

Coding at the block layer is very similar to what was discussed in Chapter 5. Again, rather than coding the actual value of DC DCT coefficients in intracoded blocks, the coefficient is predicted by the value of the previous intracoded block of the same type (i.e., Y , U , or V). The prediction difference is then transmitted. The predictor is reset at the start of a slice, when a non-intra-macroblock is decoded or when a macroblock is skipped. The last two of these only occur in P and B pictures. The value to which the predictor is reset is defined in Table 6.18. The reset value is the midpoint of the relevant DC range.

Figure 6.34 shows the prediction of DC DCT coefficients for Y , U , and V blocks. In the case of Y blocks, the top left block is predicted from the bottom right block of the previous macroblock, the top right block is predicted from the top left block, the bottom left block is predicted from the top right block, and the bottom right block is predicted from the bottom left block. For U and V blocks, the DC DCT coefficient is predicted from the U or V block in the previous macroblock.

As was the case in Chapter 5, the value of the prediction difference is coded in two parts. The first part gives the number of bits used for the prediction difference that immediately follows and is called **dct_dc_size_luminance** for luminance

Table 6.18 DC coefficient reset value for various DC coefficient precisions.

intra_dc_precision	Precision (bits)	Reset value
0	8	128
1	9	256
2	10	512
3	11	1024

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

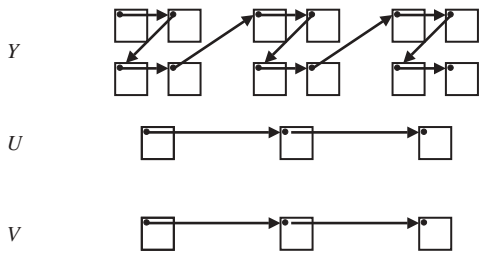


Figure 6.34 Prediction of DC DCT coefficients in intramacroblocks.

blocks and **dct_dc_size_chrominance** for chrominance blocks. The variable-length codes are given in Table 6.19.

So, a value of zero indicates that the difference has zero bits, that is, it has the value zero and so the DC DCT coefficient is the same as the relevant predictor. A value of one indicates that the prediction difference (**dct_dc_differential**) is represented by a single bit and so can take values -1 or $+1$. A value of two indicates that the prediction difference is represented by 2 bits and so can take the values -3 , -2 , $+2$, or $+3$. The full definition of **dct_dc_differential** is given in Table 6.20. The large range of possible values comes from the fact that DC coefficients can be represented by up to 11-bit accuracy (i.e., values from 0 to 2047) in MPEG-2 video. This means that the prediction difference can take values in the range of ± 2047 .

All other DCT coefficients in intrablocks as well as all coefficients in inter- or motion-compensated blocks are coded in the manner described in Chapter 5. However, there are two possible variable-length codes that can be used to encode the

Table 6.19 VLCs for **dct_dc_size_chrominance** and **dct_dc_size_luminance**.

Luminance		Chrominance	
dct_dc_size_luminance	VLC	dct_dc_size_chrominance	VLC
0	100	0	00
1	00	1	01
2	01	2	10
3	101	3	110
4	110	4	1110
5	1110	5	11110
6	11110	6	111110
7	111110	7	1111110
8	1111110	8	11111110
9	11111110	9	111111110
10	111111110	10	1111111110
11	111111111	11	1111111111

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 6.20 Definition of **dct_dc_differential**.

DC prediction difference	dct_dc_differential	Code word
−2047 to −1024	11	0000000000 to 0111111111
−1023 to −512	10	0000000000 to 0111111111
−511 to −256	9	0000000000 to 0111111111
−255 to −128	8	0000000000 to 0111111111
−127 to −64	7	0000000000 to 0111111111
−63 to −32	6	0000000000 to 0111111111
−31 to −16	5	0000000000 to 0111111111
−15 to −8	4	0000000000 to 0111111111
−7 to −4	3	0000000000 to 0111111111
−3 to −2	2	0000000000 to 0111111111
−1	1	0000000000 to 0111111111
0	0	None
1	1	1
2 to 3	2	10 to 11
4 to 7	3	100 to 111
8 to 15	4	1000 to 1111
16 to 31	5	10000 to 11111
32 to 63	6	100000 to 111111
64 to 127	7	1000000 to 1111111
128 to 255	8	10000000 to 11111111
256 to 511	9	100000000 to 111111111
512 to 1023	10	1000000000 to 1111111111
1024 to 2047	11	10000000000 to 11111111111

© This Table is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

(run, level) pairs defining nonzero DCT coefficients. Blocks not coded in intramode are always coded using the VLC codes defined in Chapter 5. Intracoded blocks can also be coded in this way if the **intra_vlc_format** flag at the picture layer is zero. An alternate VLC code word set is used for intrablocks when the **intra_vlc_format** flag is set to one. Full details of the alternate VLC code words can be found in the standard documentation.

6.4. VIDEO BUFFER VERIFIER

The video buffer verifier is a hypothetical decoder that is connected directly to the output of a compliant encoder. The input buffer to the decoder, referred to as the VBV buffer, receives coded data from the encoder. Data is removed from the buffer at regular intervals. On each occasion, the data removed is that corresponding to an entire coded picture. In order to conform to the MPEG specification, a bit stream is not permitted to cause the VBV buffer either to overflow or to underflow. As described earlier, the VBV buffer size and the bit rate of the encoder are specified at the sequence layer.

The **vbv_delay** field given in the picture header specifies the number of periods of a 90-kHz clock (which is derived from the 27-MHz system clock) that the VBV waits after receiving the final byte of the picture start code before decoding that picture.

For constant bit-rate operation,¹⁹ the value to be placed in the **vbv_delay** field can be calculated according to the following equation.

$$\text{vbv_delay}_n = \frac{90,000 \times B_n}{R}$$

where B_n is the VBV buffer occupancy immediately before removing picture n from the buffer but after removing any headers, user data, and stuffing that immediately precede the data elements of picture n and R is the actual bit rate as opposed to the value given in the sequence header that is rounded up to the next multiple of 400 bits/s.

Picture data of the n th coded picture enters the VBV buffer at a rate $R(n)$ as defined below.

$$R(n) = \frac{d_n^*}{(\tau_n - \tau_{n+1} + t_{n+1} - t_n)}$$

where d_n^* is the number of bits from the final bit of the n th picture start code to the final bit of the $(n+1)$ th picture start code, τ_n the decoding delay coded in the **vbv_delay** field for the n th coded picture in seconds, and t_n is the time when the n th coded picture is removed from the VBV buffer.

For constant bit-rate operation, the value of $R(n)$ remains constant throughout the sequence to the accuracy allowed by the quantization of **vbv_delay**.

Prior to the receipt of the first picture start code of a video sequence and following the final picture start code of a video sequence, $R(n)$ is assumed to take the value specified in the **bit_rate** field.

At the start of a video sequence, the VBV buffer is filled with all of the header data up to and including the first picture start code. The buffer is then further filled with the incoming bit stream for the time specified in the **vbv_delay** field of this first picture header at which time decoding (i.e., the removal of the first picture from the VBV buffer) begins. This is illustrated in Figure 6.35.

From the time when decoding begins, the VBV buffer is regularly examined.²⁰ At the time when the VBV buffer is examined and prior to the removal of any picture data, the total number of bits in the buffer lies between zero and the maximum VBV buffer size. All of the data for the picture that has been in the buffer for the maximum time at the time of examination is then instantaneously removed. This must not result in buffer underflow (i.e., the number of bits in the buffer cannot be negative).

The time at which the VBV buffer is examined depends upon whether the sequence is interlaced or progressive, whether the picture structure is field or frame,

¹⁹ This is the usual situation in digital television applications.

²⁰ The exact times are defined a little later.

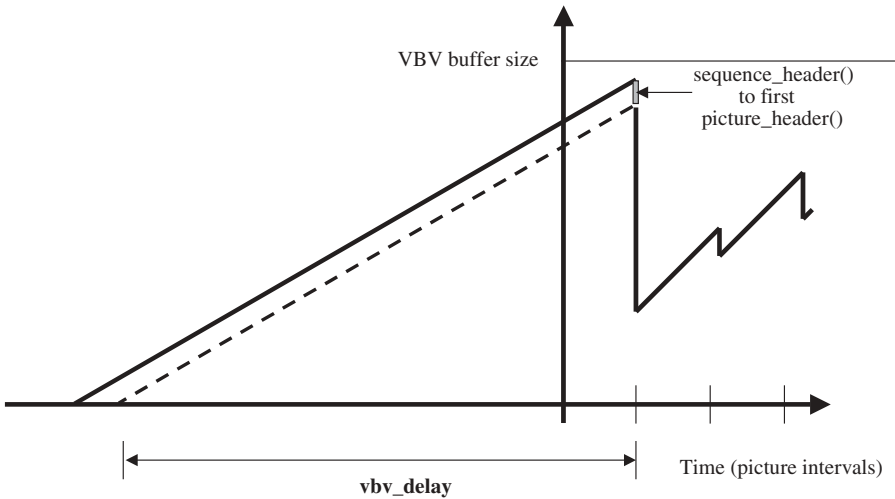


Figure 6.35 VBV buffer occupancy up to the removal of the first picture.

whether the value of the **repeat_first_field** field in the picture coding extension is set to one or not, and on the type of picture (field or frame) that is being decoded, the frame/field structure, and the state of the **repeat_first_field** flag in a previous picture.

The time for which the VBV buffer is examined is determined by the pictures in display order as opposed to the order in which they are removed from the buffer. Although B pictures are decoded and displayed immediately upon their removal from the buffer, I and P pictures are decoded immediately but may not be displayed for some time as they are transmitted before B pictures, which precede them in display order.

We use some examples to make the situation clear.²¹ Let us first consider the case of a video sequence that does not contain B pictures. In this case, the picture transmission order is the same as the picture display order. The time for which the VBV is examined is then determined by the display time of each picture. Thus, if the current picture is a field picture, the VBV is examined after one field time. Alternatively, if the current picture is a frame picture then the VBV is examined after two field times if **repeat_first_field** is not set to one and after three field times if **repeat_first_field** is set to one. In each case, the time between VBV examinations is just the display time of the current picture. This is illustrated in Figure 6.36 where it is assumed that all pictures are frame pictures and that pictures I_0 , P_2 , and P_4 have the **repeat_first_field** field set to one and so are displayed for three field times each.

The situation is slightly more complex when B pictures are included. However, the time between examinations is still determined by the display time of the picture currently being displayed. Since B pictures are displayed immediately,

²¹Only interlaced video sequences are considered here. For progressive sequences, there are no field pictures and any field repetition implies that the entire progressive picture is repeated.

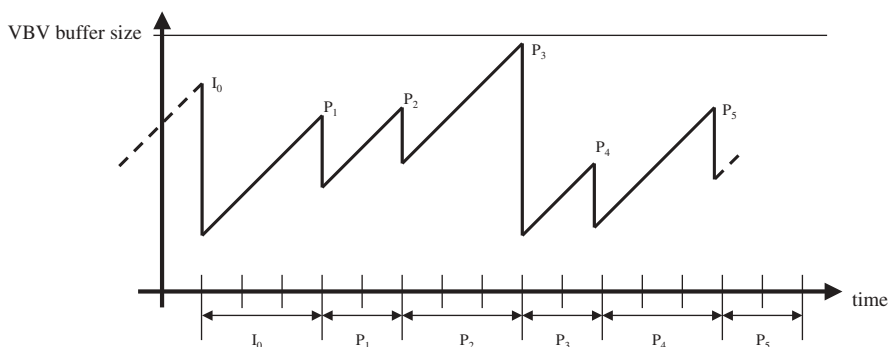


Figure 6.36 Typical VBV for interlaced video sequence without B pictures.

© This Figure is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

when a B picture is being decoded, the time between VBV examinations is one field time in the case of a field-structured B picture, two field times in the case of a frame-structured B picture when **repeat_first_field** is not set to one, and three field times in the case of a frame-structured B picture when **repeat_first_field** is set to one.

When B pictures are included, any subsequent I or P picture that is needed to predict the B pictures is transmitted before these B pictures. Thus, when an I or P picture is received, it implies that all the B pictures prior to the previously received I or P pictures have been received and decoded, and therefore, it is time for this previously received I or P picture to be displayed. The time between VBV examinations when I or P pictures are being decoded is therefore the time required to decode this previously received I or P picture. The time between VBV examinations is two field times if the previous I or P picture is frame structured and **repeat_first_field** is not set to one, three field times if the previous I or P picture is frame structured and

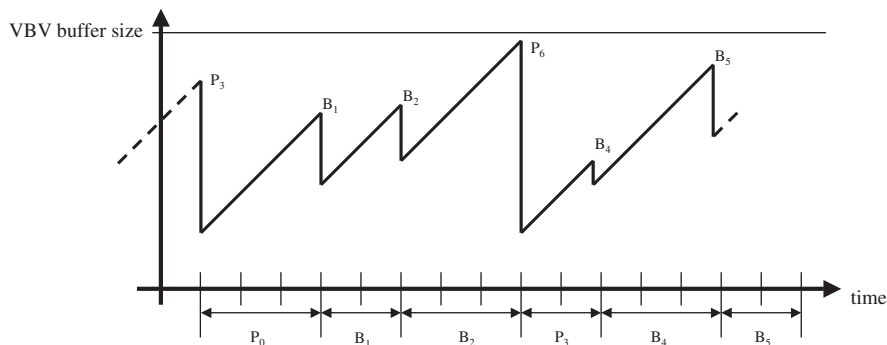


Figure 6.37 Typical VBV for interlaced video sequence with B pictures.

© This Figure is based on AS/NZS 13818.2:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

repeat_first_field is set to one, one field time if the current I or P picture is the first field of a field-structured picture, one field time if the current I or P picture is the second field of a field-structured picture and **repeat_first_field** is not set to one, and two field times if the current I or P picture is the second field of a field-structured picture and **repeat_first_field** is set to one. This is illustrated in Figure 6.37 for a simple case with only frame pictures. In Figure 6.37, P_0 , B_2 , and B_4 have display durations of three fields.

6.5. PROFILES AND LEVELS

The complete MPEG-2 video standard is designed to cater to a large number of different applications ranging from videoconferencing to high-definition television and beyond. For an encoder or decoder to fully comply with the standard, it would need to be able to handle all of these applications. Such encoders and decoders would be far more complex than necessary for their intended applications and would involve significant unnecessary expense. For this reason, MPEG has defined a number of profiles and levels for the video standard. These allow compliance with the standard at a point that is more suitable for a particular application.

A profile specifies a defined subset of the complete syntax of the standard. This means that profiles do not use all of the coding tools provided by the standard. For example, it would be possible to define a profile that did not allow temporal prediction (i.e., either interprediction or motion-compensated prediction) between pictures. Such a profile would lead to inexpensive encoders and decoders because interpicture predictions (and in particular, motion-compensated prediction) are computationally complex operations. In fact, no such profile currently exists.

A level places constraints on the values that may be taken by parameters in the bit stream. Parameters include values such as picture size, picture rate, and bit rate. Placing a limit on the maximum values of these parameters also leads to simpler encoders or decoders. For example, limiting the maximum picture size reduces the memory requirement for internal frame stores whereas reducing the bit rate reduces the speed of operation of the variable-length code encoders and decoders.

At the time of writing, MPEG has defined seven profiles and four levels. We now briefly discuss each of these profiles and levels.

6.5.1. Profiles

6.5.1.1. Simple Profile

The simple profile is intended for real-time applications. As such, round-trip delays are important, and so additional delays in encoding and decoding need to be avoided. For this reason, B pictures are not allowed in the simple profile. In addition, the simple profile uses the restricted slice structure and only the 4:2:0 color sampling structure. The simple profile is defined only for main level.

6.5.1.2. Main Profile

Main profile is the profile for digital television services. The only difference between simple profile and main profile is that main profile allows B pictures. Main profile is defined for all four levels (low, main, high 1440, and high).

6.5.1.3. Signal-to-Noise Ratio (SNR) Scalable Profile

The scalable profiles in MPEG allow a service to be made up of more than one transmitted bit stream. Receiving just the base-level bit stream provides a service of a particular quality. Receiving the base and one (or more) enhancement bit streams provides a service of enhanced quality. In the case of SNR scalability, the enhancement is in the picture quality of the service. Using this profile, a maximum of two bit streams (base plus a single enhancement) are allowed. The SNR scalable profile is defined for low and main levels.

6.5.1.4. Spatial Scalable Profile

In this case, the enhancement of quality produced by the enhancement bit stream is an increase in the spatial resolution of the displayed pictures. This might mean that the base-level service is at standard-definition television resolution whereas the enhanced service could be at high-definition television resolution. A single SNR scalable bit stream is also permitted, and this may be applied at either resolution level. The spatial scalable profile is defined only for the high-1440 level.

6.5.1.5. High Profile

The high profile is identical to the spatial scalable profile except that both 4:2:0 and 4:2:2 color sampling structures are allowed. The high profile is defined for main, high 1440, and high levels.

6.5.1.6. Professional Profile

The professional profile has been defined for use within television stations where higher quality levels are required so that subsequent postprocessing of the video material is possible. It needs to allow higher bit rates and is defined primarily for main-level applications. However, as it can encode all the active lines of video (as opposed to those defined by CCIR Recommendation 601), it does not strictly conform to any of the currently defined levels.

6.5.1.7. Multiview Profile

The multiview profile allows for the efficient coding of a service comprising more than one view such as a stereo television. It is defined for the four existing levels (low, main, high 1440, and high).

Table 6.21 Parameter values for various levels of MPEG-2.

Parameter	Low level	Main level	High-1440 level	High level
Horizontal size (pixels)	352	720	1,440	1,920
Vertical size (pixels)	288	576	1,152	1,152
Picture rate (Hz)	30	30	60	60
Pixel rate (s^{-1})	3,041,280	10,368,000	47,001,600	62,668,800
Bit rate (Mbits/s)	4	15	60	80
Bits/picture (kbits)	167	626	2,503	3,337
VBV buffer size (bits)	475,136	1,835,008	7,340,032	9,781,248
Motion vector range	-64 to +63.5	-128 to + 127.5	-128 to + 127.5	-128 to + 127.5
Intra-DC precision (bits)	8-10	8-10	8-11	8-11

6.5.2. Levels

The parameter restrictions for the various defined levels of MPEG-2 are shown in Table 6.21. For profiles that include scalability, there is a need for separate definition of some of the parameters for each bit stream (base or enhancement). As scalability is not used in current digital television services, this additional information has been omitted for clarity.

6.6. SUMMARY

This chapter has provided an overview of the MPEG-2 video compression standard that forms the basis of current digital television services worldwide. The aim of the chapter has been to introduce the reader to the main features and functionalities of the MPEG-2 standard. It has not attempted to provide sufficient depth to allow the reader to completely implement the standard. However, armed with the information contained in this chapter and a copy of the standard itself, a competent student should be capable of implementing a compliant encoder or decoder.

PROBLEMS

- 6.1** Calculate the compression ratio required to represent a 720×480 pixel, 30 frames/s interlaced service with 4:2:0 chrominance structure at a total rate of 4 Mbit/s. Repeat your calculation for a 720×576 pixel, 25 frames/s interlaced service again with 4:2:0 chrominance structure.

- 6.2** List the advantages and disadvantages of each of the picture types (I, P, and B) used in the MPEG-2 video standard.
- 6.3** Why are profiles and levels important for MPEG-2 encoder and decoder manufacturers?
- 6.4** A series of pictures in a video sequence are received by an encoder and coded as the picture types shown in Figure 6.38. Determine the order in which the pictures will be transmitted to the decoder.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
I	B	P	B	B	P	P	I	B	B	B	B	P	I	I	B	P	B	B	I

Figure 6.38 Picture types for Problem 6.4.

- 6.5** For the series of pictures and picture types given in Figure 6.38, indicate which picture(s) are used as reference picture(s) for the motion-compensated prediction of each picture.
- 6.6** A series of coded pictures is received by an MPEG-2 decoder with the picture types shown in Figure 6.39.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
I	P	B	B	B	I	P	P	B	B	I	I	P	P	B	B	I	B	B	I

Figure 6.39 Picture types for Problem 6.6.

- (a) In what order should these pictures be displayed?
- (b) What pictures are held in the decoder frame stores as each picture is received?
- (c) What pictures are used as reference pictures for each received picture?
- 6.7** Although some MPEG-2 encoders use a regular sequence of picture types for simplicity, there is no requirement for this. Thus, an intelligent encoder is able to choose the picture type according to the characteristics of each picture to be encoded.
- (a) What characteristics would the pictures have that would make them particularly suitable for encoding as a particular picture type?
- (b) Suggest some simple algorithms by which an encoder might go about making this decision.
- 6.8** In MPEG-2 video, great care is taken to ensure that not more than 23 consecutive zeros can occur. Explain briefly why this is important.
- 6.9** In the production of services for digital television, bit streams from different sources often need to be edited together.
- (a) Why is it desirable to perform editing at the bit stream level rather than decoding and then editing the decoded pictures prior to a further encoding stage?
- (b) Explain how the use of the **closed_gop** and **broken_link** flags in the GOP header can be used to assist the bit stream editing process.

- 6.10 The MPEG-2 video standard allows a number of motion vector ranges to be defined. Describe the advantages and disadvantages of using a large motion vector range.
- 6.11 Concealment motion vectors can be transmitted in intramacroblocks. Describe how these concealment motion vectors might be used if transmission errors occur.
- 6.12 The first four macroblocks after a slice header are all coded as P macroblocks with the prediction modes and motion vectors given in Table 6.22. Calculate the differential motion vector values transmitted for each macroblock.

Table 6.22 Prediction modes and motion vectors for Problem 6.12.

Macroblock	1	2	3	4
Prediction mode	Forward	Skipped	Forward	Forward
Motion vector	(−1.0, −1.0)	—	(−1.5, +1.0)	(−2.0, +2.5)

- 6.13 The first four macroblocks after a slice header are all coded as B macroblocks with the prediction modes and motion vectors given in Table 6.23. Calculate the differential motion vector values transmitted for each macroblock.

Table 6.23 Prediction modes and motion vectors for Problem 6.13.

Macroblock	1	2	3	4
Prediction mode	Backward	Interpolated	Forward	Interpolated
Forward motion vector	—	(−2.0, −4.0)	(−3.5, −4.0)	(−3.0, −3.5)
Backward motion vector	(+3.5, −2.5)	(+2.0, −2.0)	—	(+3.0, +4.5)

- 6.14 Under what circumstances is 3:2 pulldown useful in MPEG-2 video systems? How does it work and what are the advantages?
- 6.15 What is the use of pan and scan in MPEG-2 video? How is it implemented and in what circumstances is it useful?
- 6.16 Two coded macroblocks in the same slice are separated by 35 skipped macroblocks. What is the variable-length coded value of **macroblock_address_increment** that is transmitted to the decoder?
- 6.17 The field motion vector for dual-prime prediction in a P frame picture is (−8.5, + 6.5). The differential motion vector is (+0.5, +0.5). Calculate the displacement of the pixels to be used to predict each field in the macroblock. You may assume that the top field is transmitted first for each picture.
- 6.18 The first eight coded macroblocks in a particular slice taken from a P picture have the **macroblock_address_increment**, macroblock type (I or P), and raw motion vectors as shown in Table 6.24. What are the unencoded values of the motion vectors transmitted to the decoder after the appropriate motion vector prediction?

Table 6.24 Macroblock types and motion vectors for Problem 6.18.

Macroblock address increment	Macroblock type	Raw motion vector
3	P	(+7.5, -9.0)
2	P	(+8.0, -9.5)
1	I	—
1	P	(+6.0, -5.0)
2	P	(+6.0, -5.0)
1	I	—
1	P	(+6.0, -5.0)
1	P	(+6.0, -5.0)

6.19 The first eight coded macroblocks in a particular slice taken from a B picture have the **macroblock_address_increment**, macroblock type (I, B forward prediction, B backward prediction, and B interpolated prediction), and raw motion vectors as shown in Table 6.25. What are the unencoded values of the motion vectors transmitted to the decoder after the appropriate motion vector prediction?

Table 6.25 Macroblock types and motion vectors for Problem 6.19.

Macroblock address increment	Macroblock type	Raw forward motion vector	Raw backward motion vector
3	B interpolated	(+4.5, -5.0)	(-3.5, +4.0)
2	B forward	(+4.0, -6.5)	—
1	B backward	—	(-4.5, +3.5)
1	B backward	—	(-5.0, +2.0)
2	B interpolated	(+5.0, -5.0)	(-1.0, +3.5)
1	I	—	—
1	B backward	—	(0.0, +1.0)
1	B forward	(+6.0, -5.0)	—

6.20 The DC coefficients of the first two macroblocks of an I picture are shown below. Calculate the variable-length code words transmitted to the decoder to represent each of these coefficients. Assume that DC coefficients are represented to 10-bit accuracy.

Macroblock 1	Macroblock 2
$Y_1 = 643$	$Y_1 = 685$
$Y_2 = 659$	$Y_2 = 732$
$Y_3 = 801$	$Y_3 = 847$
$Y_4 = 894$	$Y_4 = 922$
$U = 140$	$U = 184$
$V = 278$	$V = 278$

MATLAB EXERCISE 6.1: BIDIRECTIONAL MOTION-COMPENSATED PREDICTION

In this exercise we study the effectiveness of bidirectional motion-compensated prediction.

Section 1 Forward and backward prediction

Write a MATLAB program that will perform forward motion estimation and backward motion estimation from a current picture to a past and a future picture, respectively. Use minimum absolute error as the matching criteria and generate a motion-compensated prediction picture from both the past and future pictures.

It is now necessary to decide whether forward or backward prediction is to be employed. Do this by choosing as the predictor the forward or backward prediction macroblock that has the minimum-squared error to the macroblock in the current picture.

Summarize your results under the following circumstances:

- The past, current, and future pictures are all from the same video sequence.
- The past picture is from one video sequence and the current and future pictures are from a different video sequence. This simulates a cut prior to the current picture.
- The past and current pictures are from one video sequence whereas the future picture is from a different video sequence. This simulates a cut after the current picture.

Consider the circumstances where an inappropriate prediction (i.e., forward instead of backward) is made. Suggest and apply tests that might improve the accuracy of this decision.

Section 2 Interpolative prediction

Repeat Section 1 of this exercise, but this time allow for interpolated prediction as well as forward and backward predictions. The interpolated prediction is formed by averaging the forward and backward motion-compensated prediction pictures.

In what circumstances is interpolative prediction useful?

MATLAB EXERCISE 6.2: DUAL-PRIME MOTION-COMPENSATED PREDICTION

In this exercise you will implement dual-prime motion-compensated prediction and compare its performance to other forms of motion-compensated prediction.

Section 1 Implementation of dual-prime motion-compensated prediction

Using the description contained in the chapter, implement dual-prime motion-compensated prediction for frame pictures. Use a full search motion estimation approach with a search range from -8.0 to $+7.5$ pixels and for each field motion vector. For each possible field motion vector you should consider each of the nine possible differential motion vectors.

Test your dual-prime program on the first two pictures of a video sequence. Compare the result obtained with that obtained using frame-based motion-compensated prediction with the same search range.

If time permits, also experiment with pictures from other video sequences.

Section 2 Speeding up dual-prime motion-compensated prediction

Rather than considering the differential motion vector in association with each possible field motion vector, simply calculate the best field motion vector and then calculate the appropriate differential vector just for this “best” field motion vector. Comment on the change in performance.

MATLAB EXERCISE 6.3: FIELD AND FRAME MOTION-COMPENSATED PREDICTION

In this exercise we will consider the advantages of using adaptive field/frame motion-compensated prediction in frame pictures.

Section 1 Comparison of field- and frame-based motion-compensated prediction

Use MATLAB to implement frame-based motion-compensated prediction and field-based motion-compensated prediction on a pair of frame pictures. Choose the prediction type according to which of the two produces the lower value of squared error for each macroblock in the current picture.

Experiment with pictures from the available sequences, and comment on the characteristics of macroblocks that require field as opposed to frame coding.

Section 2 Realistic field/frame decision

The use of field motion-compensated prediction carries a penalty in that two motion vectors are required as opposed to a single motion vector for frame motion-compensated prediction. For this reason, the decision whether to use field or frame-based prediction should be biased slightly toward frame prediction.

One way to do this is to introduce a threshold so that if

$$\text{MSE}_{\text{frame}} < \text{MSE}_{\text{field}} + \text{threshold}$$

then frame prediction is employed. Incorporate this threshold into the simulation of Section 1 of this exercise and comment on the results achieved.

MATLAB EXERCISE 6.4: FIELD AND FRAME DCT CODING

In this exercise, we examine the implementation of adaptive field/frame DCT coding of pictures both with and without motion-compensated prediction.

Section 1 Field and frame DCT

Develop MATLAB code to implement both field-based DCT and frame-based DCT.

It is now necessary to find a criterion to determine whether to use field- or frame-based DCT. One simple approach would be to look at the amount of energy packed in low-frequency DCT coefficients in each case. Use the energy in the low-frequency 4×4 DCT coefficients of each block in a macroblock to determine whether the macroblock should be coded in field or frame mode.

Section 2 Field/frame decision prior to DCT

In the development of the MPEG-2 video standard, a decision was made in the pixel domain as to whether frame- or field-based DCT should be employed. The method employed is described below.

1. Determine the sum-squared difference between lines 1 and 3, 2 and 4, 3 and 5, . . . 14 and 16 and call this var1,
2. Determine the sum-squared difference between lines 1 and 2, 2 and 3, 3 and 4, . . . 14 and 15 and call this var2.
3. If var1 is less than var2, then use field DCT coding. Otherwise use frame DCT.

This is a simple method for estimating whether correlation is greatest between every second line (as would be expected in macroblocks suited to field DCT) or between adjacent lines (as would be the case in macroblocks suited for frame DCT).

Implement this approach and comment on its ability to correctly choose between frame and field DCT.

Section 3 Incorporation of motion-compensated prediction

The previous two sections have concentrated on intra macroblocks only. Extend your simulation to include field and frame motion-compensated prediction as well. MATLAB Exercise 6.3 provides details on how to do this.

There are now four possible types of macroblocks:

- Frame motion-compensated prediction with frame DCT coding.
- Frame motion-compensated prediction with field DCT coding.
- Field motion-compensated prediction with frame DCT coding.
- Field motion-compensated prediction with field DCT coding.

What characteristics do macroblocks that fall under each of these categories possess?